3. Process Management

Operating Systems (OS)
Master in Innovation and Research in Informatics (MIRI)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026 Q1

FIB

Contents

- Introduction
- Process scheduling
- Process resilience
- **Virtualization**

Contents

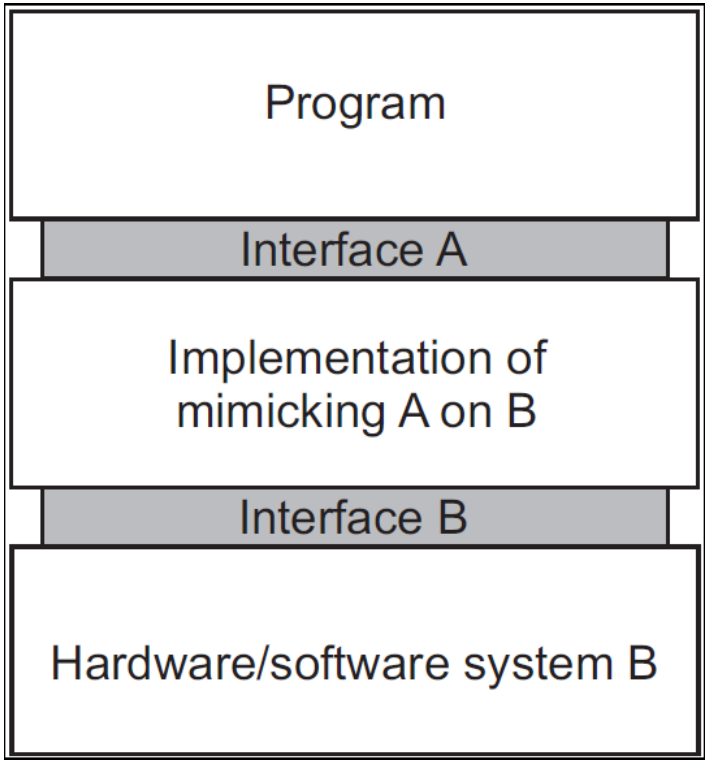
- Introduction
- Process scheduling
- Process resilience
- **Virtualization**
  - **Introduction**
  - Hardware virtualization
  - OS-level virtualization
  - Virtual clusters and resource management

Introduction

- **Virtualization** provides interesting features:
  - Creates the illusion of multiple dedicated systems on the same physical system
  - Isolates programs from the underlying system and from other programs
  - Virtual environments can be created and destroyed readily and with little overhead
  - Facilitates the portability of programs
  - Allows agile and fine-grain dynamic resource provisioning
  - Enables fault tolerance for programs

Introduction

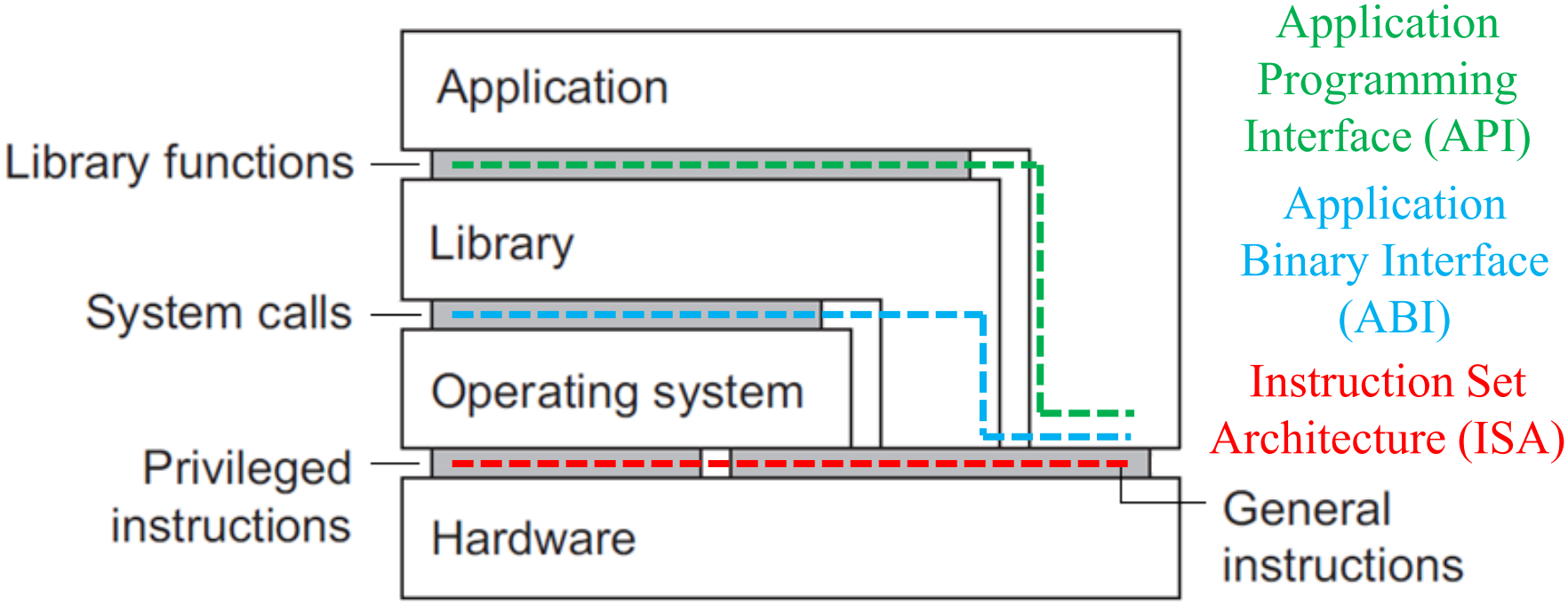
- Virtualization maps the interface and visible resources of a system onto the interface and resources of an underlying, possibly different, system
  - Introduced in 1970s to run legacy software on newer mainframe HW



Courtesy of [TaSt23]

Introduction

- Computers offer different abstraction levels
- Two types of virtualization depending on the abstraction level where virtualization occurs

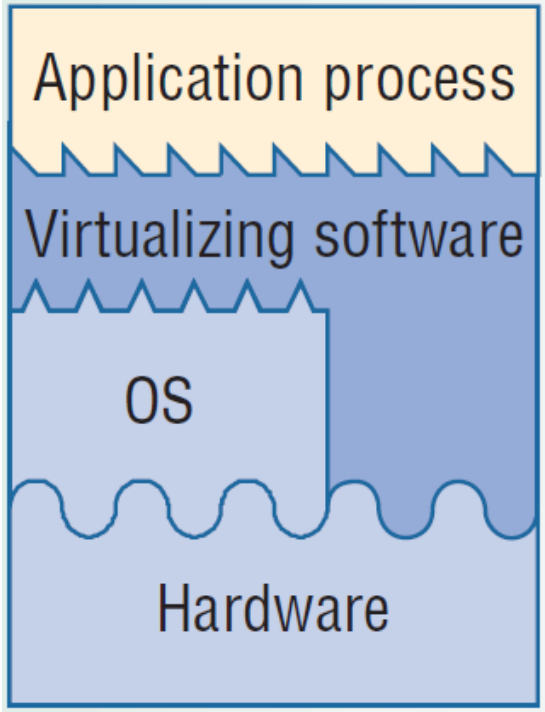


Courtesy of [TaSt23]

Types of virtualization

A. Process virtualization

- Provides a virtual environment for one (or a group of) process(es) to isolate from the underlying HW or OS and/or from other processes
- Virtualization occurs at the ABI or API interfaces: emulate both user-level instructions (†) and either system or library calls
  - (†) Might include the translation of the user instructions into the (different) ISA of the native host



Courtesy of [SmNa05]

Process virtualization

(*) a) **OS-level virtualization** (a.k.a. containers)

- Segregate and isolate multiple user-space instances (groups of processes) within a single OS
- e.g. LXC, Docker, OpenVZ, Podman

b) Application-level virtualization (a.k.a. programming-environment virtualization)

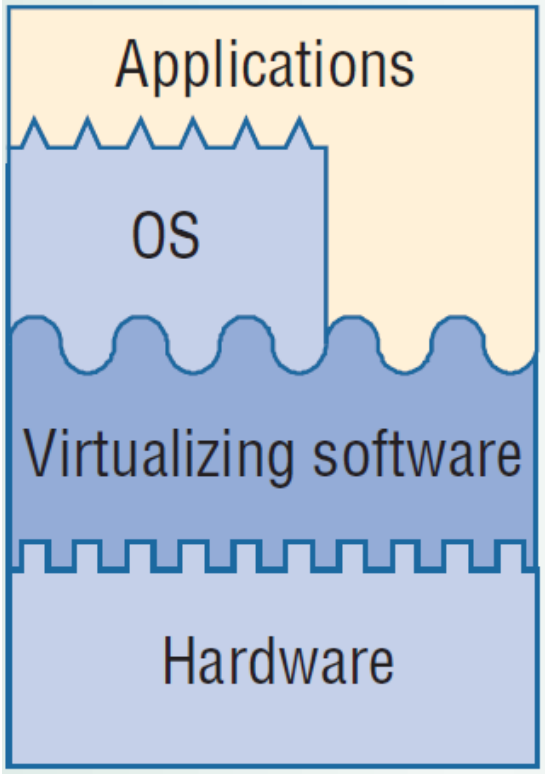
- Programming environment that abstracts details of the underlying HW or OS and allows a program to execute in the same way on any platform
- e.g. Java Virtual Machine (JVM)

(*) covered in this lesson

Types of virtualization

(*) B. System (or HW) virtualization

- Provides a complete system environment with access to virtual HW resources to allow a guest OS along with its processes to run in isolation in a shared host
- Virtualization occurs at the ISA interface: emulate ISA (†) and virtualize HW resources
- e.g. KVM, VMware, VirtualBox, Xen
- Includes full virtualization and paravirtualization



(*) covered in this lesson

Courtesy of [SmNa05]

READING REPORT

[Agache20] Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., Popa, D., *Firecracker: Lightweight Virtualization for Serverless Applications*. Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI'20), Santa Clara, CA, USA, February 25-27, 2020, pp. 419-434

Contents

- Introduction
- Process scheduling
- Process resilience
- **Virtualization**
  - Introduction
  - **Hardware virtualization**
  - OS-level virtualization
  - Virtual clusters and resource management

Contents

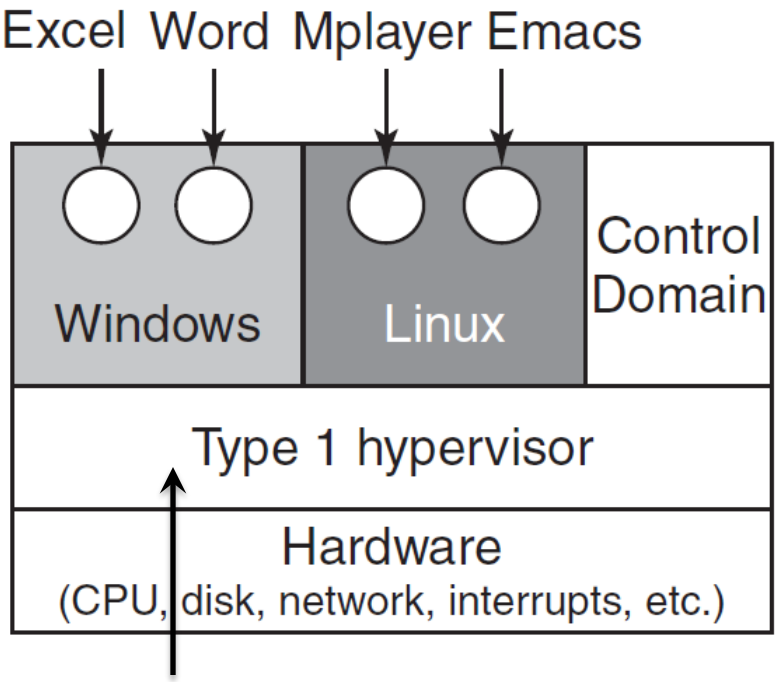
- **Virtualization**
  - Introduction
  - **Hardware virtualization**
    - **Full virtualization**
    - Paravirtualization
    - Virtualization of OS functions
  - OS-level virtualization
  - Virtual clusters and resource management

Full virtualization

- Add a software hypervisor to emulate enough hardware to allow an unmodified guest OS to run in isolation
  - Hypervisor a.k.a. Virtual Machine Monitor (VMM)
- VMM creates the illusion of multiple (virtual) machines on the same physical hardware
  - Virtualizes all the physical resources for each VM
  - Multiplexes running VMs across the physical resources of the machine
- Two types of hypervisors: Type 1 and Type 2

Bare-metal hypervisor (Type 1)

- Runs directly on top of the HW, acting as OS
- Only program running in the most privileged mode
- Must provide virtualization and OS functionality (CPU scheduling, memory & I/O management, device drivers for the HW)
- Typically deployed on server systems and production environments

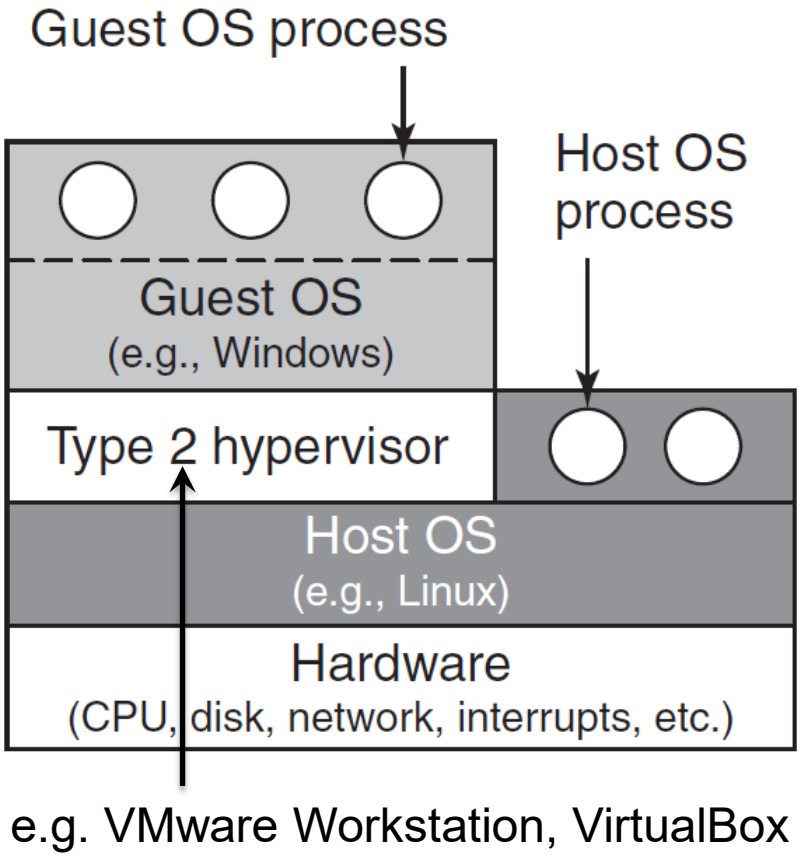


e.g. VMware ESX, Xen, Hyper-V

Courtesy of [TaBo23]

Hosted hypervisor (Type 2)

- Runs as a user-level process on top of an existing OS
- Depends on host OS for much of its functionality
- Host OS layer adds extra latency: less performance
- Typically deployed on commodity devices and development/testing environments



e.g. VMware Workstation, VirtualBox

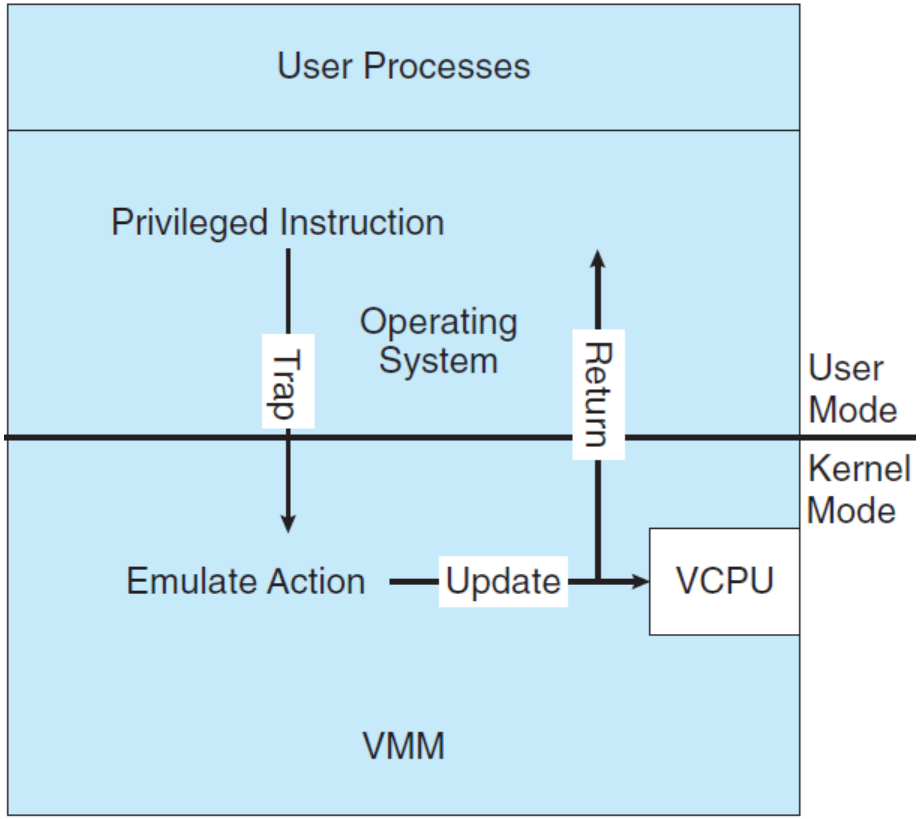
Courtesy of [TaBo23]

Requirements for full virtualization

- Safety: Only the VMM should manage the HW. Guest is limited to its virtual space
  - Instructions issued by guest OS to control the HW (a.k.a. sensitive instructions) must be emulated
- Fidelity: Programs on VM execute identically to their execution on HW (except for timing)
  - Emulation is transparent to the guest OS (it thinks that the instruction took place on the real HW)
- Performance: Non-sensitive instructions can be executed directly (if ISA is the same)

Trap-and-emulate

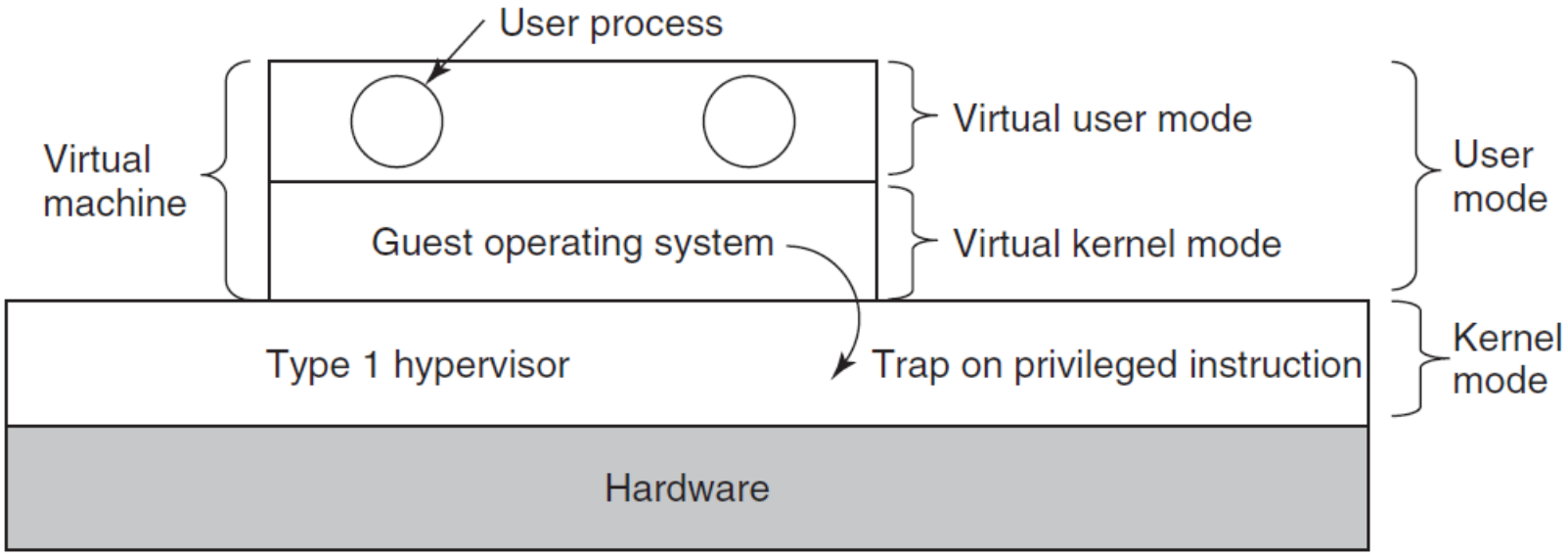
- Privileged instructions issued by the guest OS generate a trap when executed in user mode
- VMM intercepts traps from the de-privileged guest, and emulates the trapping instruction against the virtual machine state
- Expensive: trapping + emulation



Courtesy of [SiGG18]

Trap-and-emulate

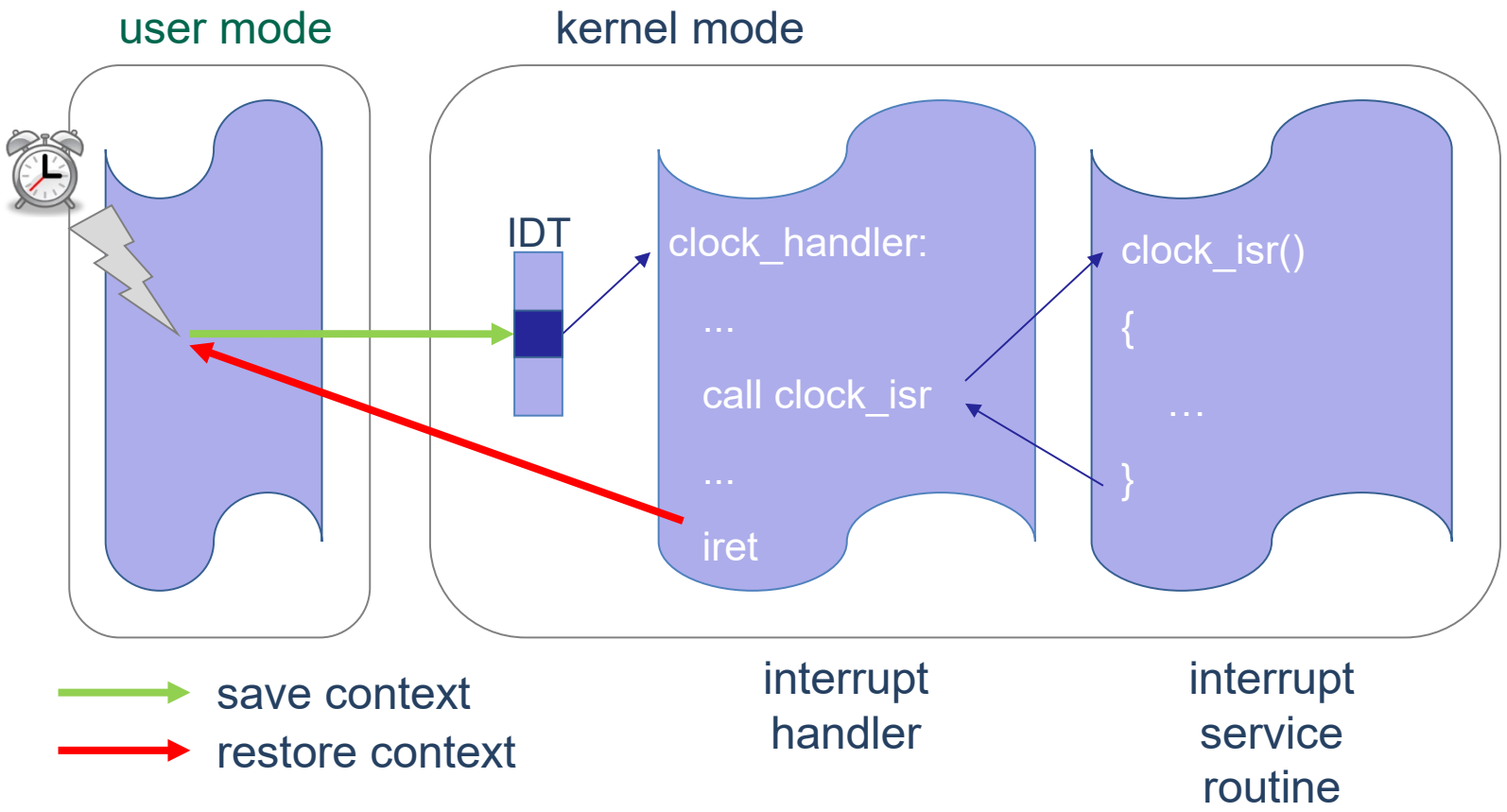
- Capturing traps generated by the guest OS also allows the VMM to transfer the guest from virtual user mode to virtual kernel mode



Courtesy of [TaBo23]

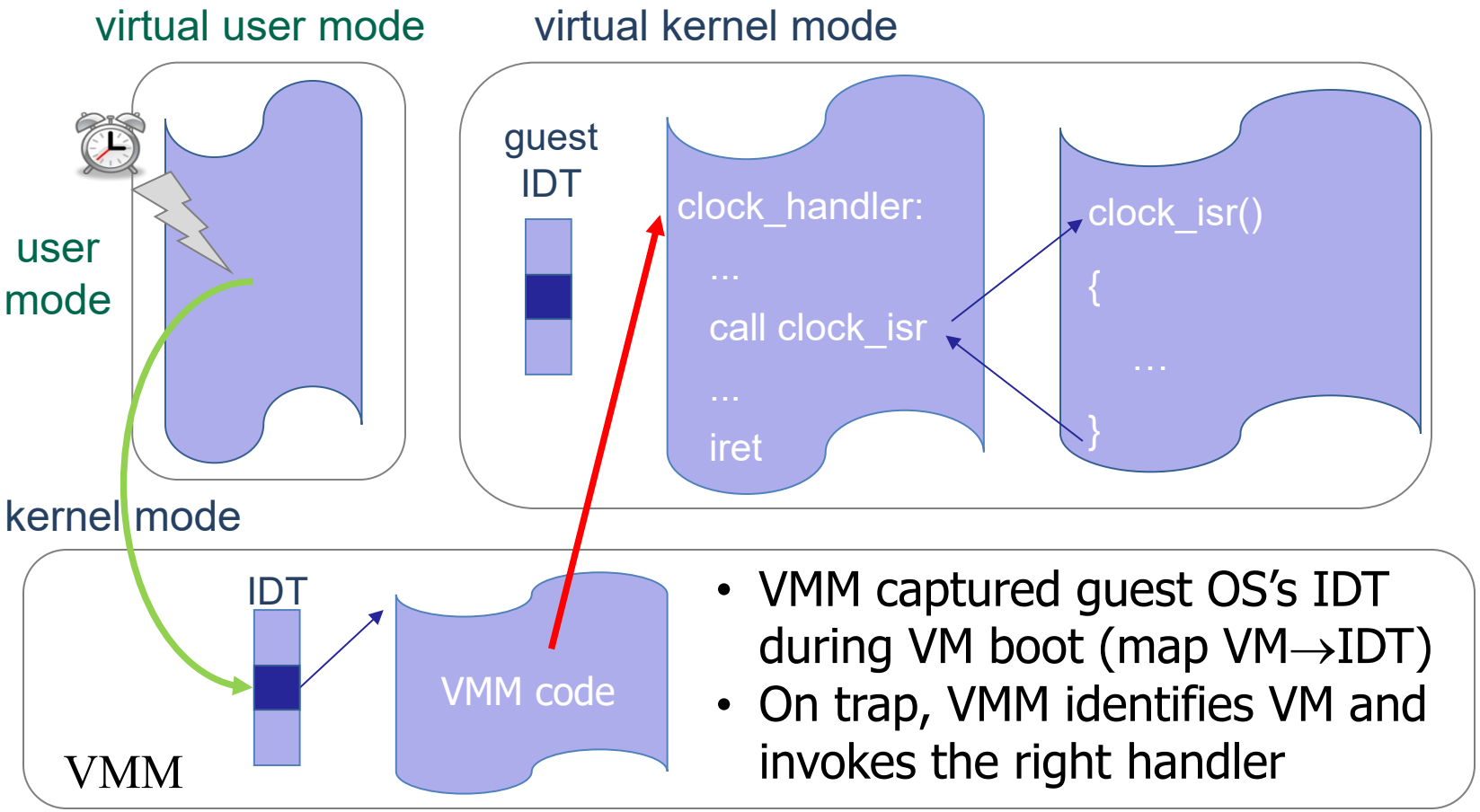
Trap-and-emulate

- Let's recall the access to kernel mode ...



Trap-and-emulate

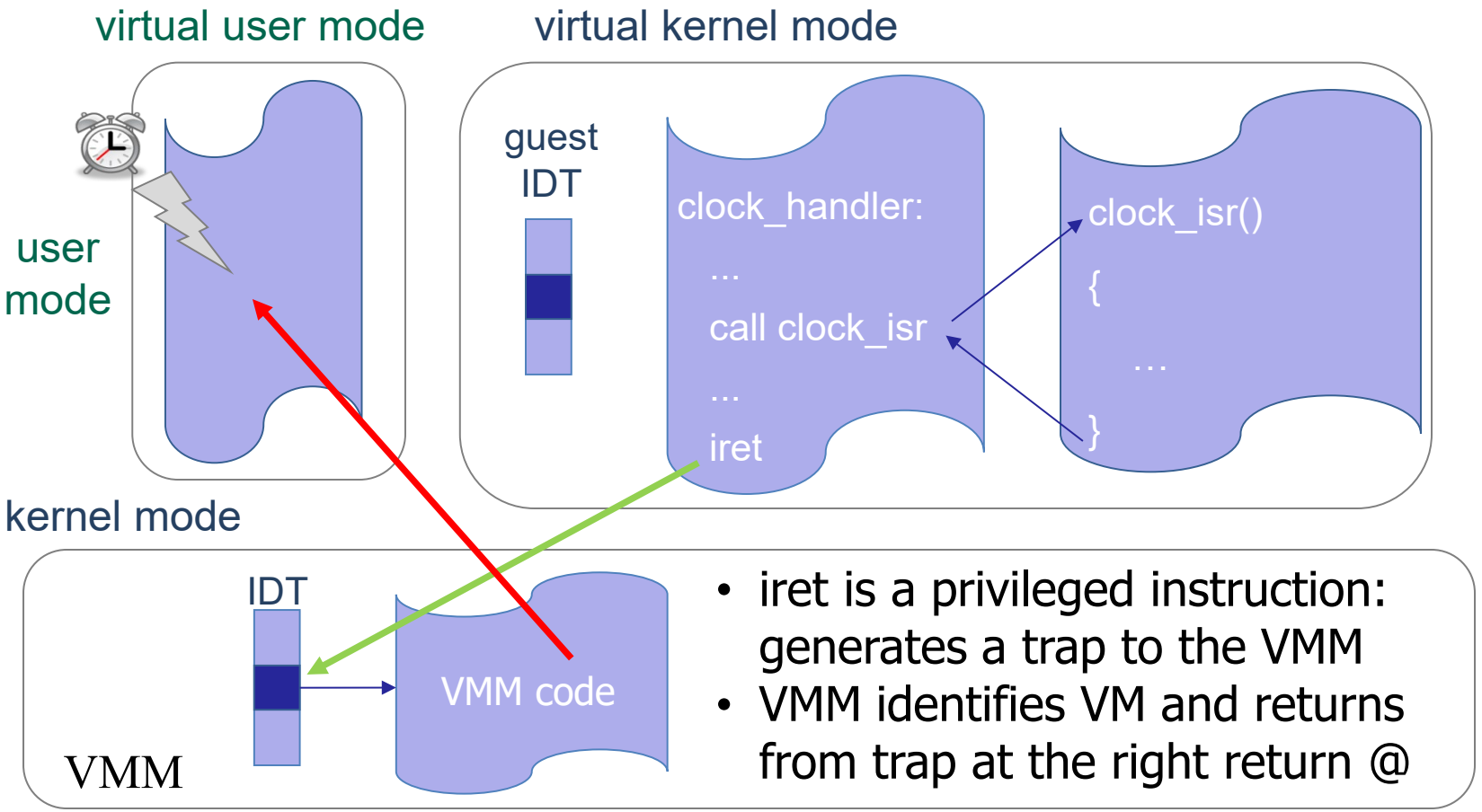
- Access to virtual kernel mode with virtualization



- VMM captured guest OS's IDT during VM boot (map VM→IDT)
- On trap, VMM identifies VM and invokes the right handler

Trap-and-emulate

- Return to virtual user mode with virtualization

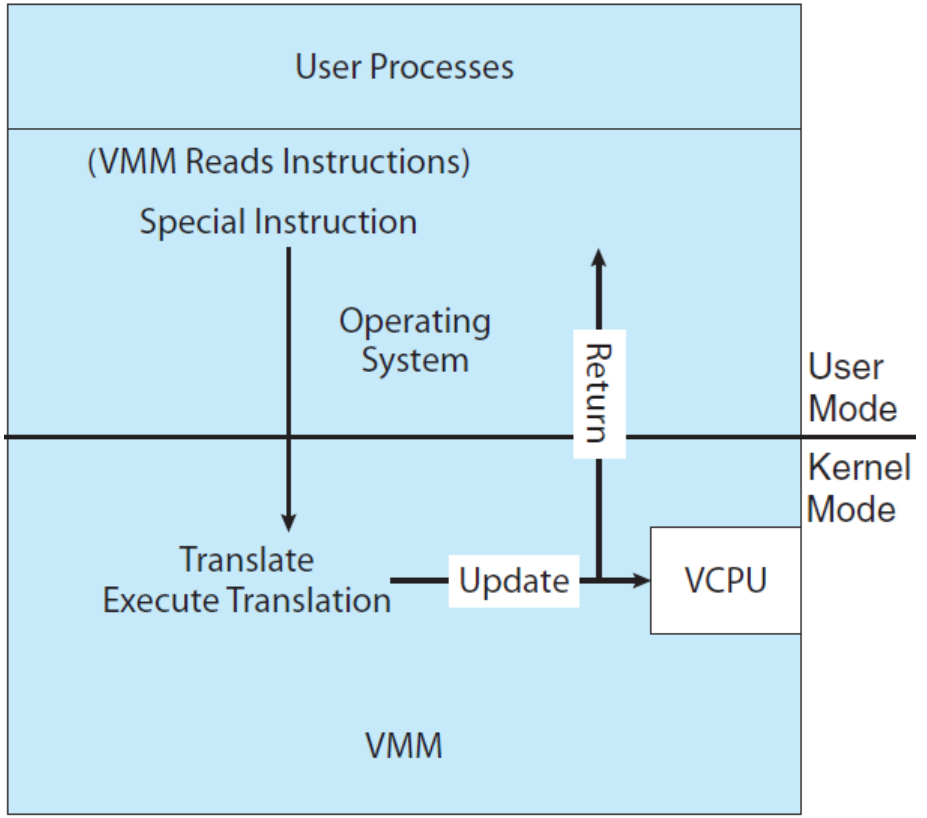


Full virtualization of Intel x86

- Trap-and-emulate needs sensitive instructions to be a subset of privileged instructions
- x86 architectures are not virtualizable using the classical trap-and-emulate technique
  - Some sensitive instructions do not trap when run in user mode (e.g. `popf`) ⇒ full list of instructions
- How was this limitation overcome?
  1. At first, with binary translation + protection rings
  2. Later on, virtualization extensions were added to the processor ⇒ HW-assisted virtualization
     - Intel VT-x (2005); AMD-V (2006)

Binary translation

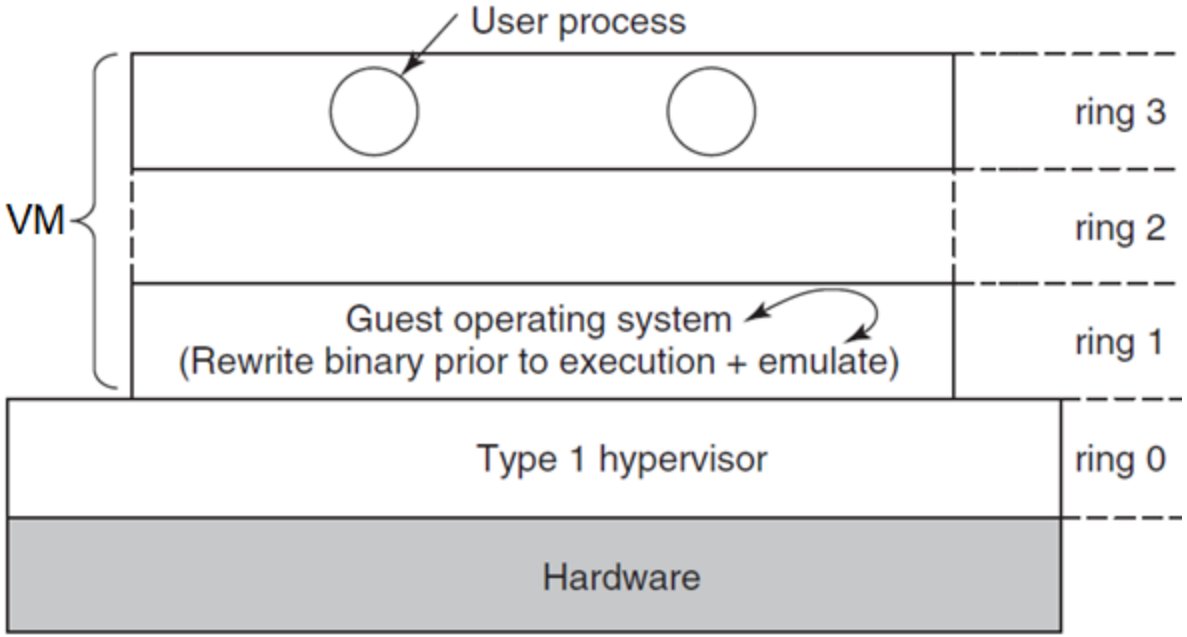
- Code executed in virtual user mode is directly executed on the CPU
- Code executed in virtual kernel mode is scanned on the fly for basic blocks
- Blocks are rewritten to replace sensitive instructions with new code that emulates their effect against VM state
- Blocks can be cached



Courtesy of [SiGG18]

Protection rings

- As x86 supports 4 protection modes (rings), the guest OS can be placed in ring 1



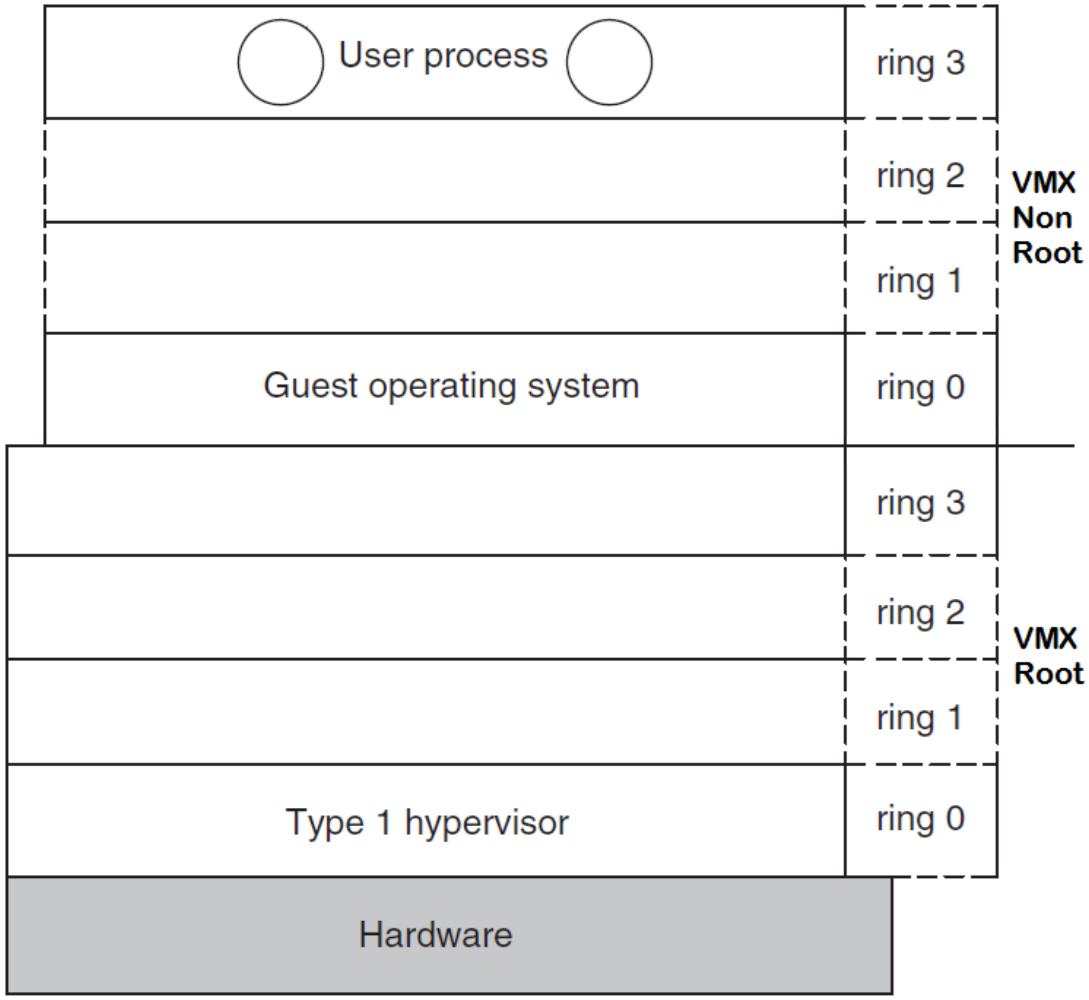
As ring 3 > ring 1, any attempt to access the guest OS memory from a user process will lead to an access violation (*)

As ring 1 > ring 0, privileged instructions within the guest OS will still trap to the VMM

(*) Note that if the system has only two modes, the guest OS runs in user mode and must use memory protection to prevent the access of OS data structures from user processes

Hardware-assisted virtualization

- Add a new VMX mode of operation
  - VMM runs in **VMX root** operation
  - Guest OS runs in **VMX non-root** operation
    - Guest OS can remain in ring 0



Courtesy of [TaBo23]

Hardware-assisted virtualization

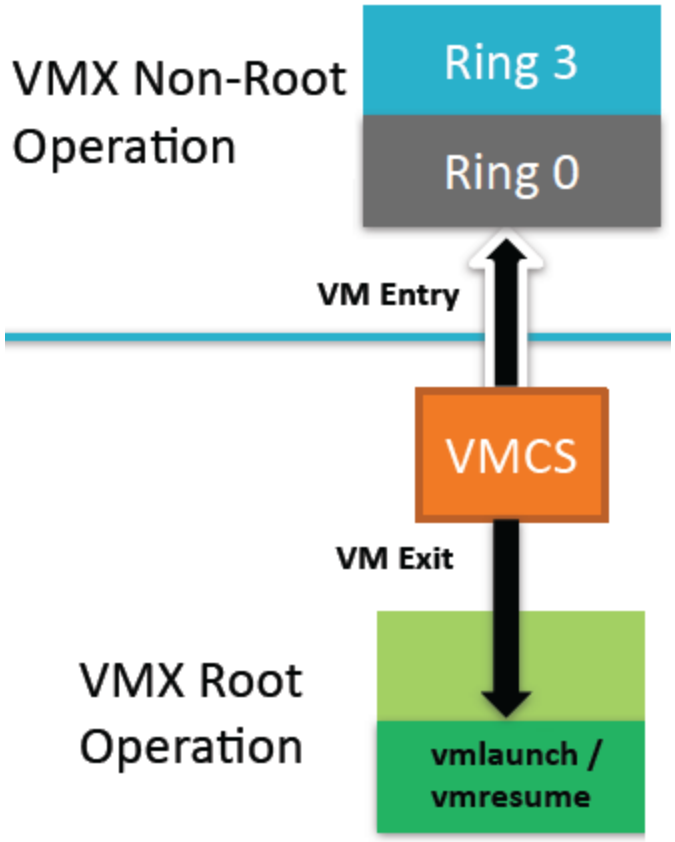
- VMX transitions

1. VM Entry

- Transitions into VMX Non-Root operation (from VMM to guest)
- Entered by VMM by using new instructions:
  - *vmlaunch* used for initial entry
  - *vmresume* used subsequently

2. VM Exit

- Transitions into VMX Root operation (from guest to VMM)
- Entered on traps from guest OS

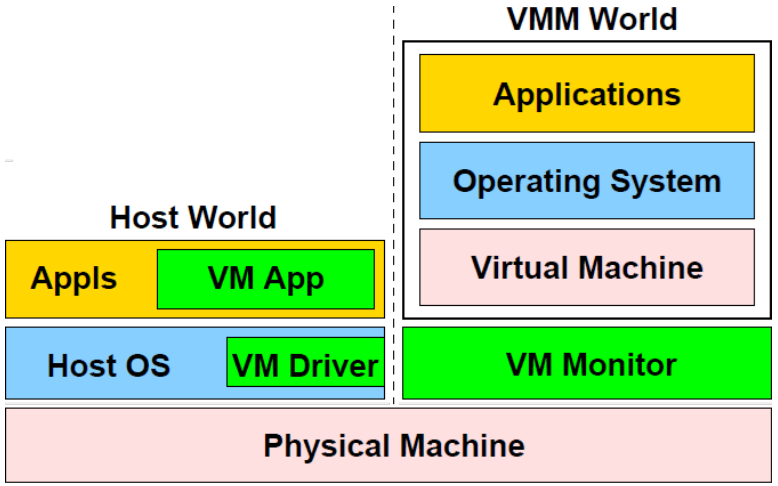


Hardware-assisted virtualization

- VM Control Structure (VMCS): data structure to manage VMX mode and VMX transitions
  1. Store guests and host states to speedup VMX transitions
     - *Guest-state area:* save there the guest-state on VM exits and load it from there on VM entries
     - *Host-state area:* load host-state from it on VM exits
  2. Control the behavior of some instructions in VMX mode (when must VM Exits occur?)
     - In VMX mode all sensitive instructions can be set to trap if executed in Non-Root operation
     - Classical trap-and-emulate becomes possible on x86!

Full virtualization in hosted VMMs

- 'Pure' hosted VMMs cannot manage HW directly as they run as user-level processes
  - Must emulate completely the HW and resort to binary translation to capture sensitive instructions
- Most modern hosted VMMs have a kernel module operating in the host OS (ring 0) that allows:
  - Manipulating the HW with privileged instructions
  - Installing trap handlers in the kernel and use classical trap-and-emulate approach
  - Switching between Host and VMM worlds: **world switch**



Courtesy of [SuVL01]

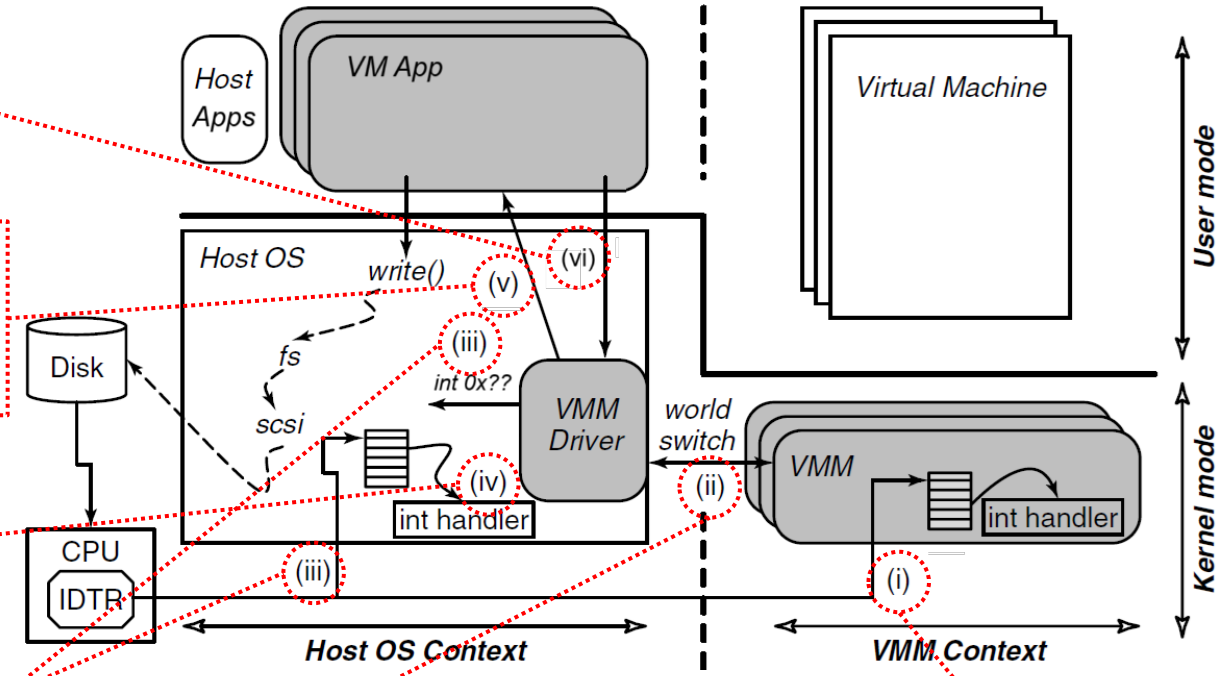
Virtualization in VMware Workstation

- Example: a disk interrupt signaling the completion of a write happens while a VM is executing (in VMM world)

(vi) VM App can resume the VM by doing a call to VMM Driver, which will generate a world switch back to the VMM context

(v) After the interrupt completes, execution resumes into VMM Driver, which returns to VM App. Being at user mode, host OS may reschedule Apps

(iv) The handler specified in the host IDT runs through its logic, calling the appropriate device driver to manage the interrupt



(iii) The switch restores IDTR to point to host IDT. VMM Driver regenerates the disk interrupt by software

(ii) As VMM cannot handle I/O directly (it has no device drivers), the handler triggers a world switch to the host OS

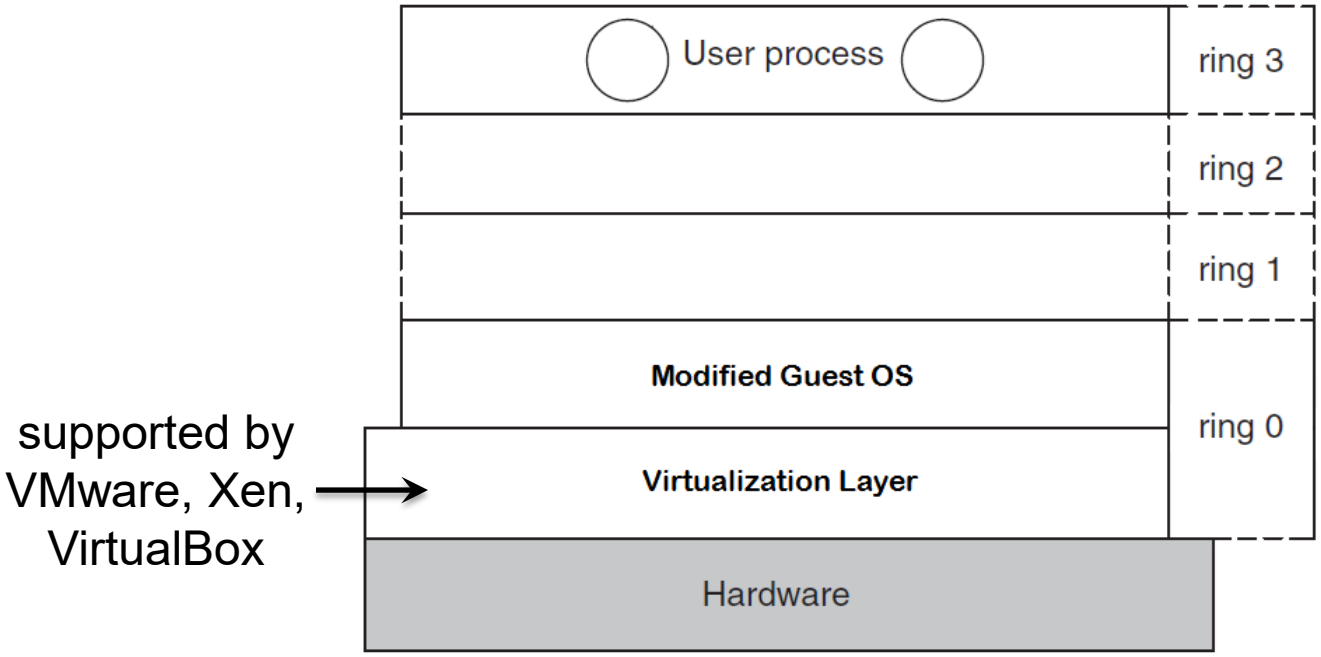
(i) HW generates the interrupt. IDTR points to the VMM external interrupt handler

Contents

- **Virtualization**
  - Introduction
  - **Hardware virtualization**
    - Full virtualization
    - **Paravirtualization**
    - Virtualization of OS functions
  - OS-level virtualization
  - Virtual clusters and resource management

Paravirtualization

- Modify source code of the guest OS to replace sensitive instructions with hypercalls to the VMM
  - ↑ Lower overhead than full virtualization
  - ↓ Guest OS tied to VMM: use VMM-agnostic hypercalls



Contents

- **Virtualization**
  - Introduction
  - **Hardware virtualization**
    - Full virtualization
    - Paravirtualization
    - **Virtualization of OS functions**
  - OS-level virtualization
  - Virtual clusters and resource management

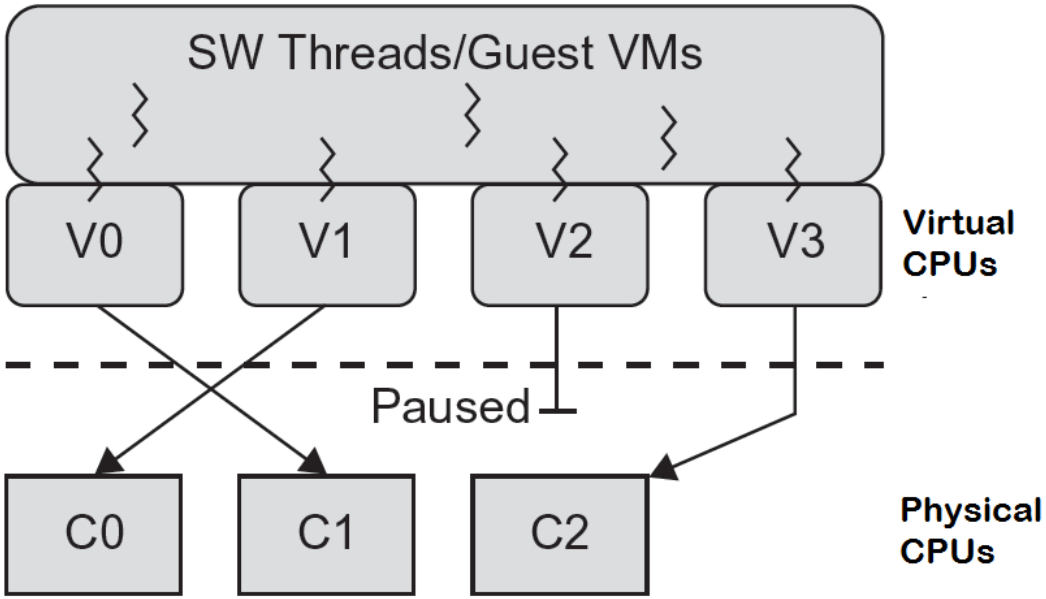
Contents

- **Virtualization**
  - Introduction
  - **Hardware virtualization**
    - Full virtualization
    - Paravirtualization
    - **Virtualization of OS functions**
      - **VCPU scheduling**
      - Memory virtualization
      - I/O virtualization
  - OS-level virtualization
  - Virtual clusters and resource management

VCPU scheduling

- Each guest is assigned one or more vCPUs
- **Double** scheduling:

1. Guest schedules processes onto the allocated vCPUs
2. VMM schedules vCPUs onto the physical CPUs



Courtesy of [HwDF11]

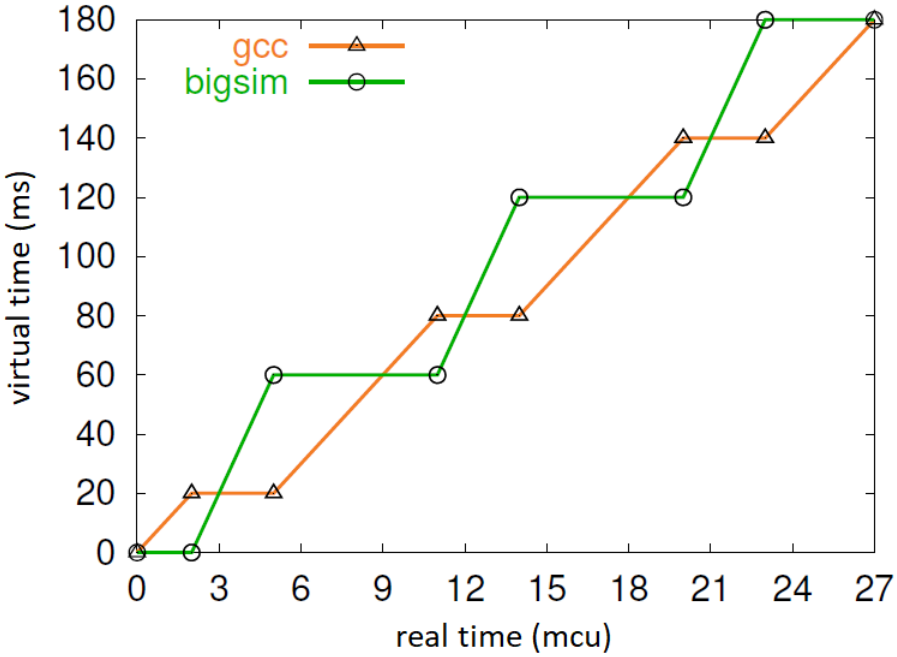
VCPU scheduling

- Scheduling at the VMM layer is typically based on proportional-share scheduling
  - Better to ensure that all VMs can at least make progress (fair CPU sharing, no starvation)
- Case study: **Borrowed Virtual Time (BVT)**
  - Allocates CPU time in proportion to the weight (CPU share) that a vCPU is assigned
  - Employs the notion of virtual time
    - Gives all runnable vCPUs equal amounts of virtual time
      - Always runs the vCPU with lowest virtual time
    - Virtual time is consumed inversely proportional to weight

Borrowed Virtual Time (BVT)

- For each $vCPU_i$, track its virtual time (A_i)
  - A_i += $RealRunningTime \times m_i$
    - Real running time measured in multiples of the *minimum charging unit (mcu)*, typically the timer interrupt period
    - m_i set proportional to $\mathbf{1/w_i}$ (being w_i the weight of $vCPU_i$), and normalized to be an integer
- Switch from **i** to **j** if $A_j \leq A_i - C \times m_i$
  - C is the *context switch allowance*
    - Real time by which the current vCPU is allowed to go beyond another runnable vCPU (it is a multiple of the mcu)
  - Prevent ping-pong switches on every timer interrupt

Borrowed Virtual Time (BVT)



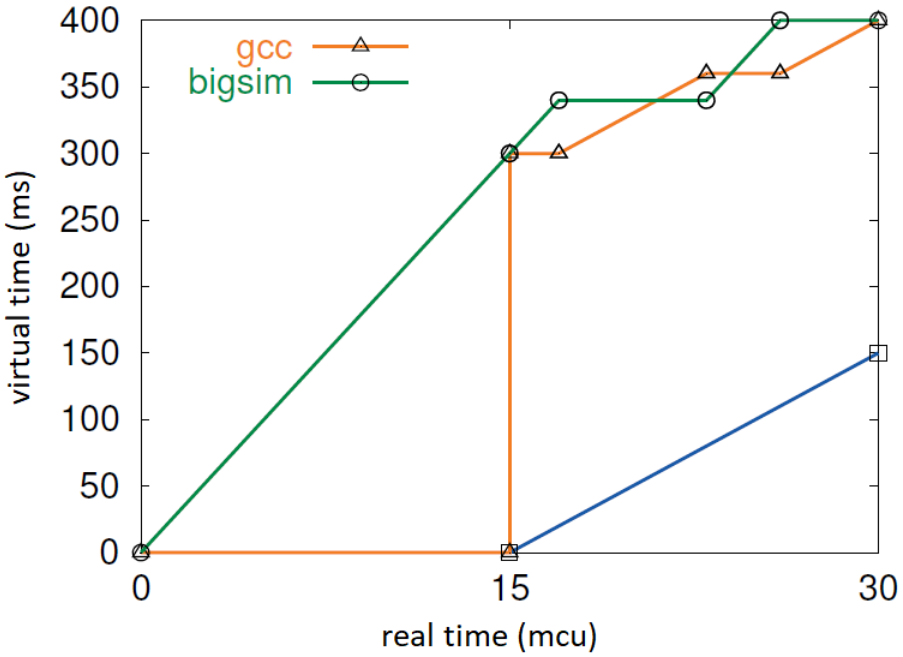
- weights: $w_{gcc} = 2/3$ ($m_{gcc} = 1$)
  $w_{bigsim} = 1/3$ ($m_{bigsim} = 2$)
- mcu = 10 ms, C = 2 × mcu = 20 ms
- CPU-bound tasks, no I/O
- diagonal line: running vCPU
- flat line: waiting (runnable) vCPU
- lines intersect where precise sharing is achieved

- Each vCPU receives its fair share: gcc uses 2/3 of the real CPU time (2+6+6+4=18) while bigsim uses 1/3 (3+3+3=9)
- All runnable vCPUs get equal amounts of virtual time, but gcc consumes its virtual time more slowly
- vCPUs always run for C time after lines intersect

Courtesy of [DuCh99]

Borrowed Virtual Time (BVT)

- $VCPU_i$ must increase its A_i after waking up from voluntary sleep $\Rightarrow A_i = \max(A_i, SVT)$
  - Otherwise vCPUs with very low A_i would starve everyone
  - Scheduler Virtual Time (SVT): $\min(A_j)$ of any runnable $vCPU_j$

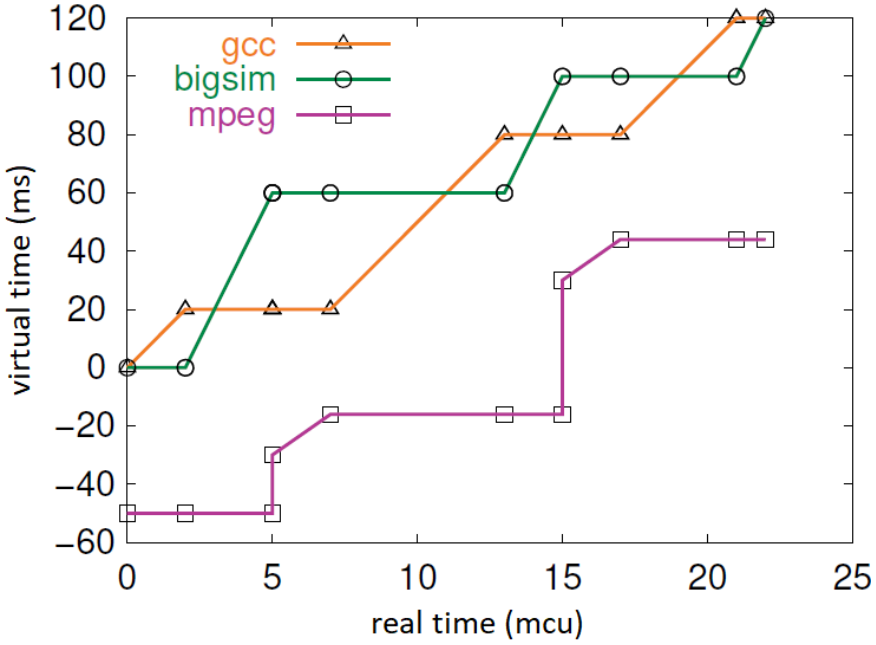


- gcc sleeps from time 0 to time 15
- A_{gcc} is brought up to SVT on wakeup
  - otherwise, would be at lower (blue) line and starve bigsim

Courtesy of [DuCh99]

Borrowed Virtual Time (BVT)

- Support time-critical tasks through **warp** values
  - Calculate effective virtual time as $A_i - (warp_i \; ? \; W_i : 0)$
    - W_i value allows borrowing virtual time from future allocation
    - $warp_i$ flag to enable/disable warp. Also disabled if task runs warped for more than a (configurable) time limit



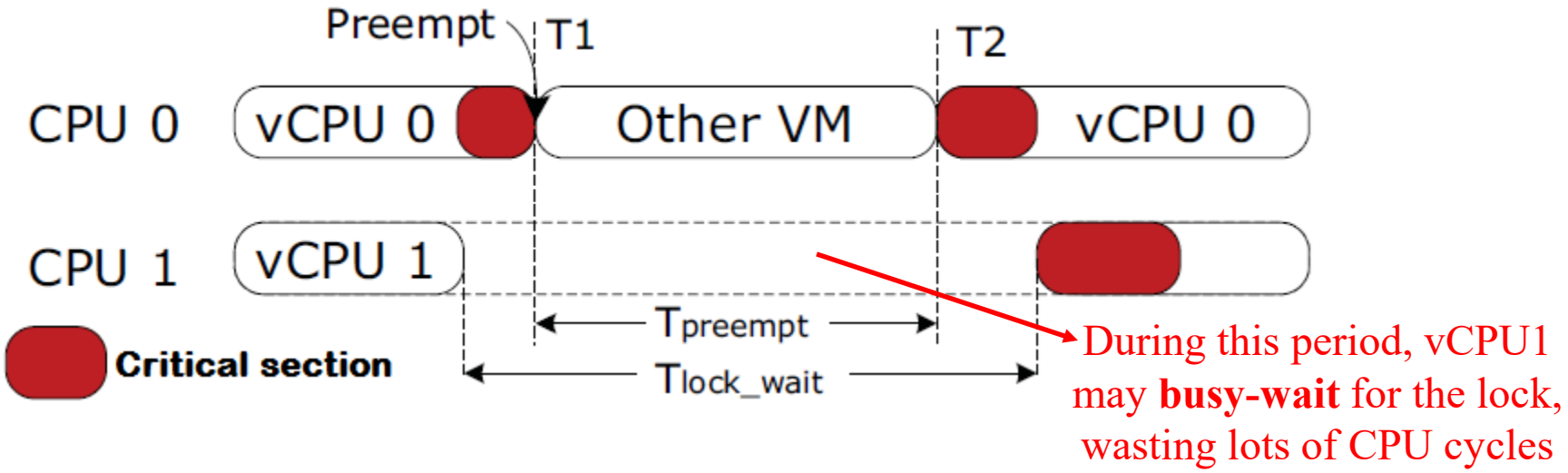
- mpeg wakes up at time 5 and time 15, runs briefly, and goes back to sleep
- mpeg runs with $W_{mpeg} = 50$, giving it the lowest effective virtual time
  - gets CPU whenever it is runnable
- like any task, its A_{mpeg} is advanced to SVT on waking up

Courtesy of [DuCh99]

VCPU scheduling

- Double scheduling worsens the Lock Holder Preemption (LHP) problem:

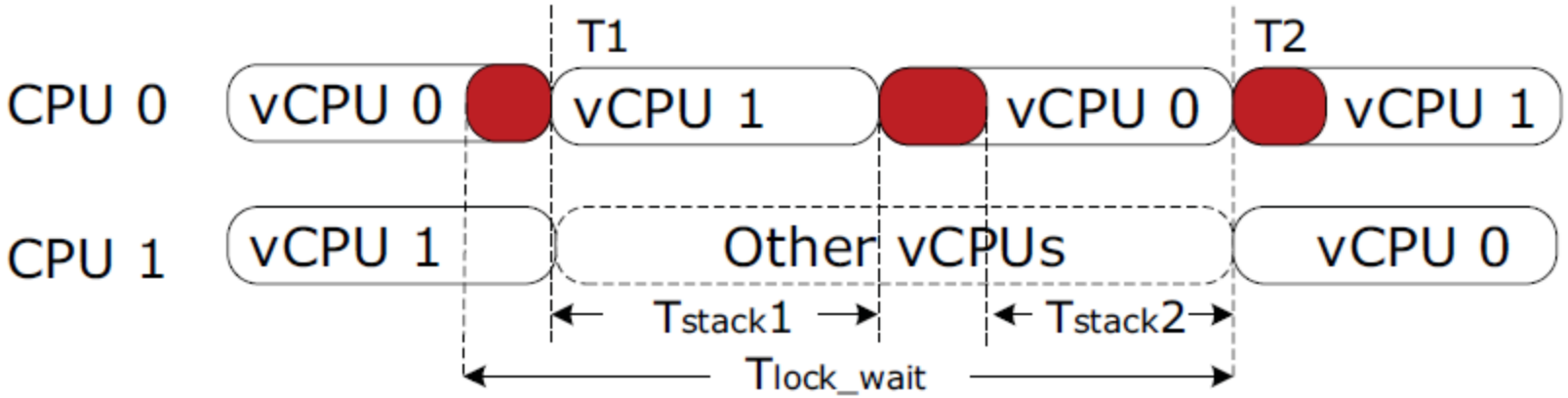
A. When vCPUs are preempted
   - Preemption of lock holder vCPU0 extends the time spent by vCPU1 waiting for the lock by $T_{preempt}$



VCPU scheduling

B. When vCPUs are stacked

- As vCPUs can be scheduled on any CPUs, the lock waiter vCPU1 could be scheduled before the lock holder vCPU0 on the same CPU, extending by $T_{stack1}+T_{stack2}$ the time spent waiting for the lock



Courtesy of [SSCZ13]

VCPU scheduling

- Several approaches to deal with LHP problem

A. Pause-Loop Exiting (PLE)
  - Hardware support to detect busy-waiting vCPUs by monitoring the number of `pause` instructions
  - VMM gets trapped so it can preempt busy-waiting vCPU and yield to another one

↓ Which vCPU should we yield to?
  - Aim for the current lock holder, but PLE events are not enough to determine who is

VCPU scheduling

B. Paravirtualized spinlocks
   - Modify the spinlock implementation in the guest OS to add hypercalls to the VMM

   a) To inform the VMM that a lock is acquired so that it prevents the preemption of the vCPU

   b) To yield to another vCPU when lock cannot be acquired quickly (instead of spinning)
      - We can yield to the most appropriate vCPU, as the VMM can track the current lock holder thanks to the hypercalls

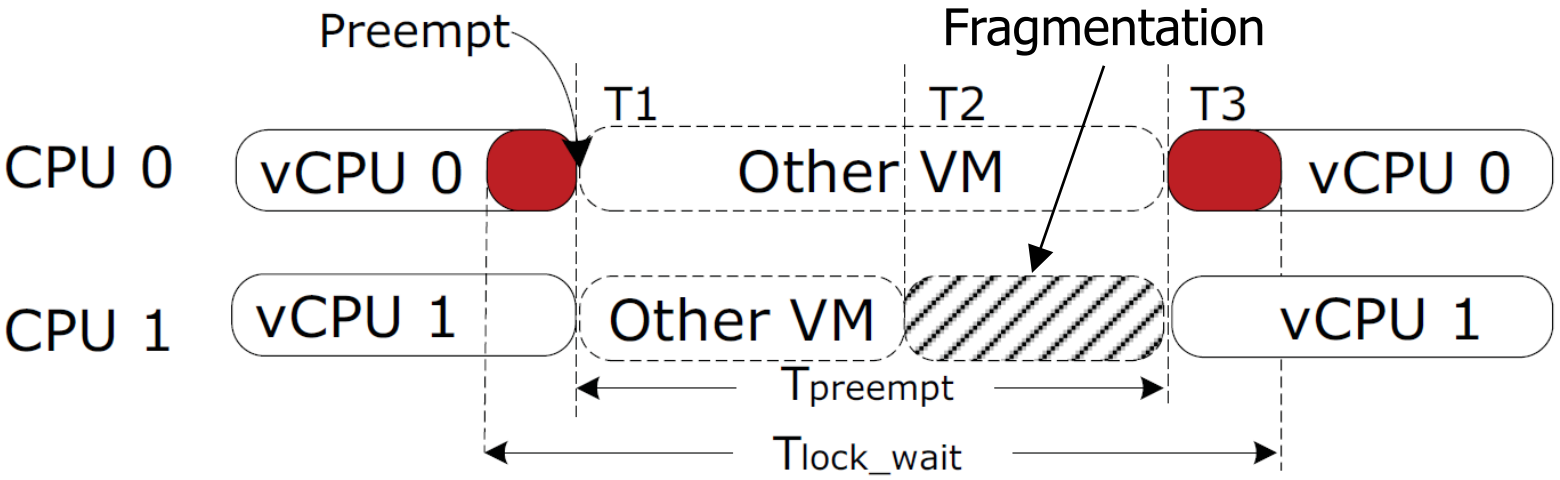
   ↓ Guest OS must be modified

VCPU scheduling

C. Gang scheduling (scheduling vCPUs of the same VM simultaneously) (*) if VM's #vCPUs ≤ #CPUs

↑ Avoids vCPU stacking (*) and busy-waiting

↓ Fragmentation, priority inversion (a higher priority task is scheduled after a lower priority one)

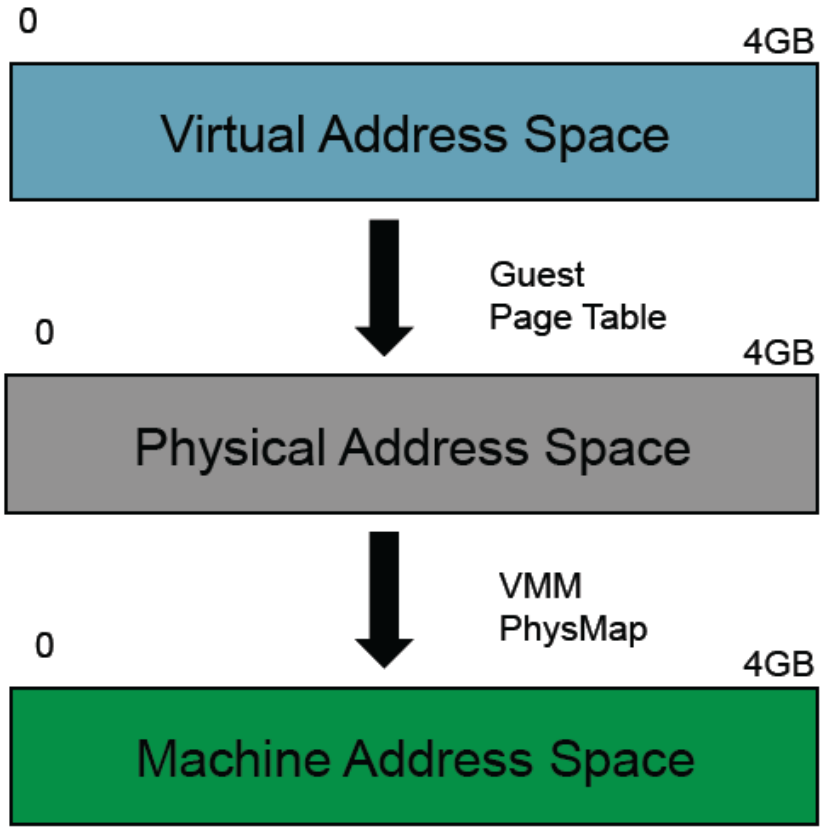


Contents

- **Virtualization**
  - Introduction
  - **Hardware virtualization**
    - Full virtualization
    - Paravirtualization
    - **Virtualization of OS functions**
      - VCPU scheduling
      - **Memory virtualization**
      - I/O virtualization
  - OS-level virtualization
  - Virtual clusters and resource management

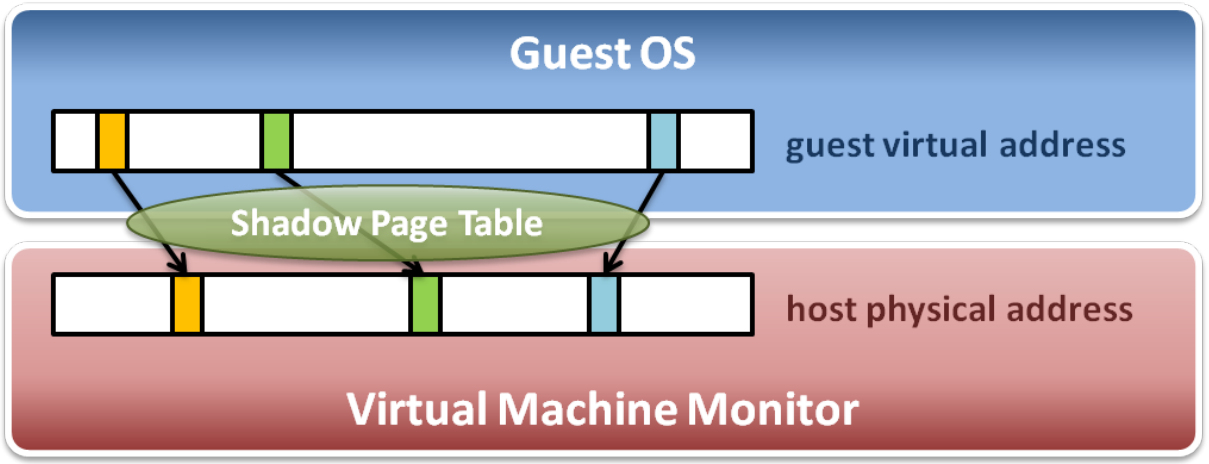
Memory virtualization

- Guest OS controls the mapping of virtual-to-physical addresses via its per-process page tables
- For each guest OS, VMM must map the guest physical memory to the underlying machine physical memory



Memory virtualization

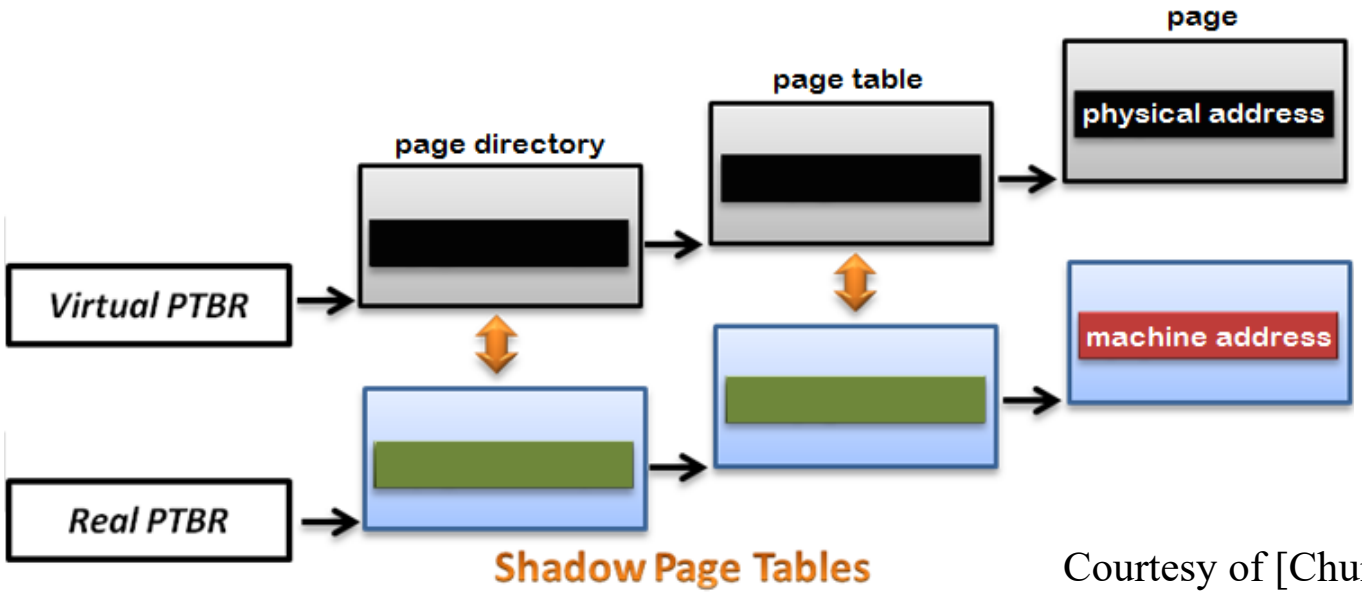
- HW MMU offers a single level of address translation, but virtualization needs two levels
- VMM must maintain **Shadow Page Tables** for each guest OS to map guest virtual addresses **directly** to machine addresses
  - HW MMU walks through the shadow page tables



Courtesy of [Chun17]

Memory virtualization

- VMM maintains a shadow page table for each guest page table and a virtual Page Table Base Register (PTBR) for each guest OS
  - Real PTBR points to the shadow page tables of the currently active guest OS



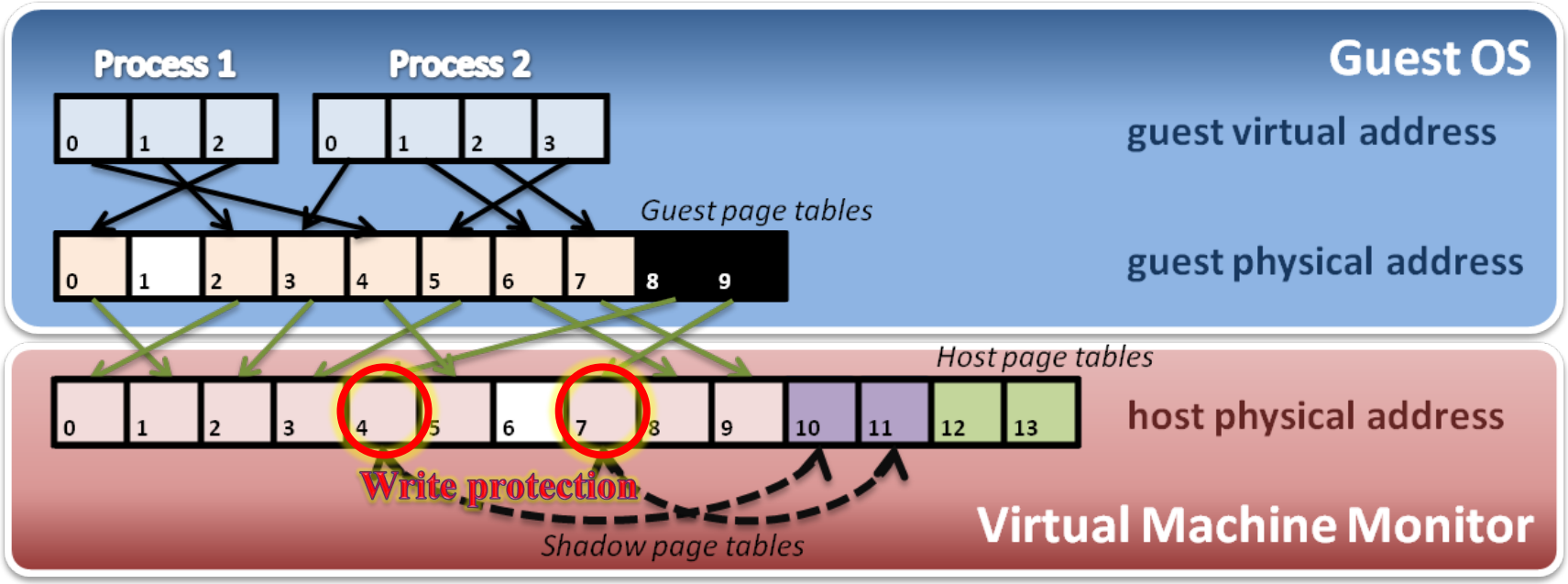
Courtesy of [Chun17]

Memory virtualization

- VMM must capture guest's attempts to install or update guest page tables ⇒ keep shadow page tables in sync
  - Full virtualization:
    - Instructions to modify the virtual PTBR are privileged, so they trap to the VMM
    - Guest page tables are **write-protected**, so their modification by the guest OS traps to the VMM
  - Paravirtualization: use hypercalls to inform the VMM about changes in the page tables
  - ↓ Maintaining shadow page tables is expensive
  - ↓ Space overhead due to shadow copying

Memory virtualization

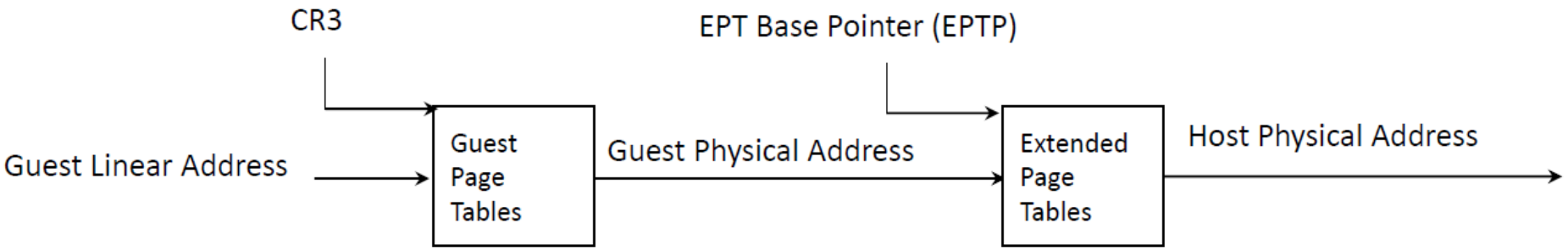
- Guest OS maintains its own page table for each process
- VMM maps each guest physical page to machine physical page
- VMM creates shadow page tables for each guest page table
- VMM protects the frames which contain guest page tables



Courtesy of [Chun17]

HW-assisted memory virtualization

- EPT (Intel) / NPT (AMD)
- HW MMU offers two nested levels of address translation:
  1. Guest page tables (maintained by the guest OS) are used to obtain guest physical addresses
  2. Extended page tables (maintained by the VMM) map guest physical addresses to host addresses



HW-assisted memory virtualization

- HW MMU can walk directly (without software intervention) through guest and extended page tables to translate guest virtual addresses to host addresses

↑ Better performance as avoids shadow paging

  - No VMM involvement to maintain tables in sync
  - Less space needed: one EPT per guest OS instead of a shadow page table per process

↓ Address translation takes longer than with the original HW, so TLB misses are more costly

Contents

- **Virtualization**
  - Introduction
  - **Hardware virtualization**
    - Full virtualization
    - Paravirtualization
    - **Virtualization of OS functions**
      - VCPU scheduling
      - Memory virtualization
      - **I/O virtualization**
  - OS-level virtualization
  - Virtual clusters and resource management

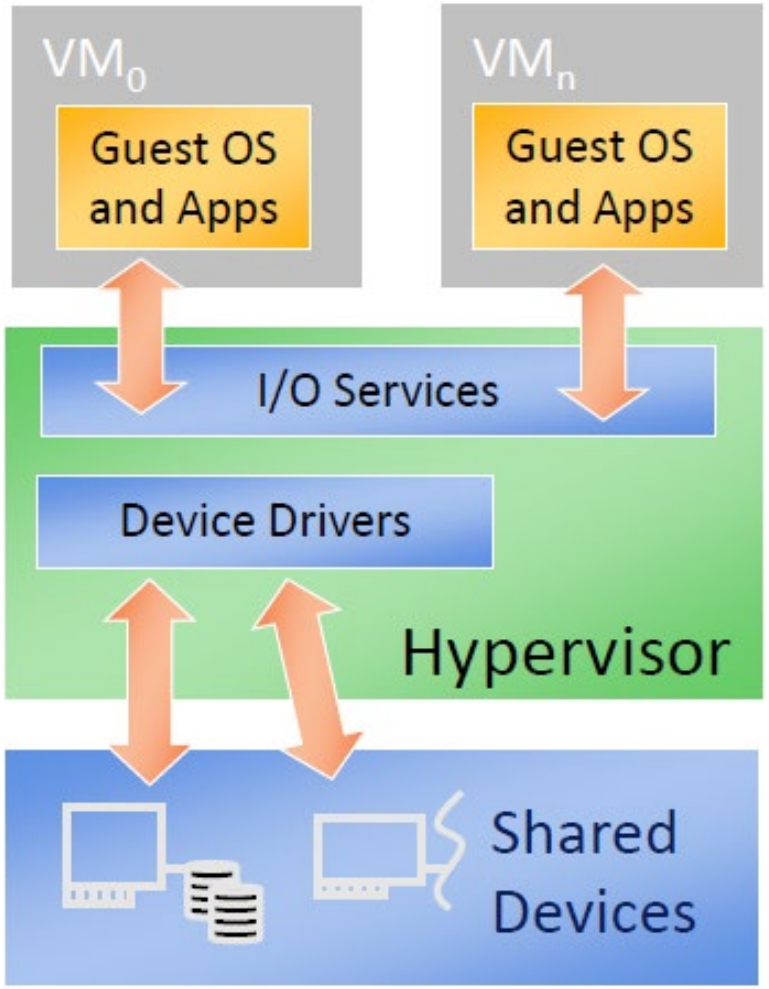
I/O virtualization

- Decouple virtual from physical I/O activity through a software indirection layer

1. Guest OS is offered with virtual I/O devices
   - Typically, well-known devices
2. VMM intercepts guest OS accesses directed at the virtual devices
   - Must prevent guest from accessing real devices
3. VMM emulates virtual I/O devices (using the physical hardware if needed)
   - Sustain illusion that guest can access real devices

Hypervisor direct model

- Virtual devices that emulate all the functions of a device are located in the VMM
- ↑ Device sharing
- ↑ Easy VM migration
- ↓ Emulation overhead
- ↓ Complex hypervisor
- ↓ Exposed to driver faults



Courtesy of [Chun17]

Service-VM/Hosted model

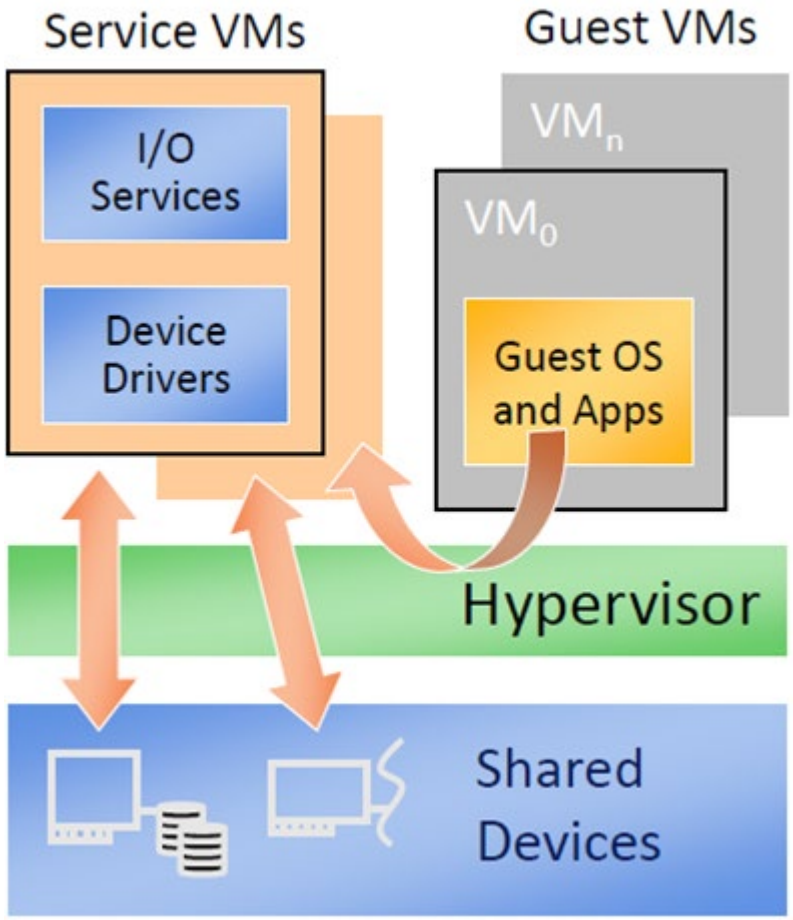
- Virtual devices run in a privileged VM or host OS
- Redirect all I/O calls from other guests to it

↑ Higher security

↑ Device sharing

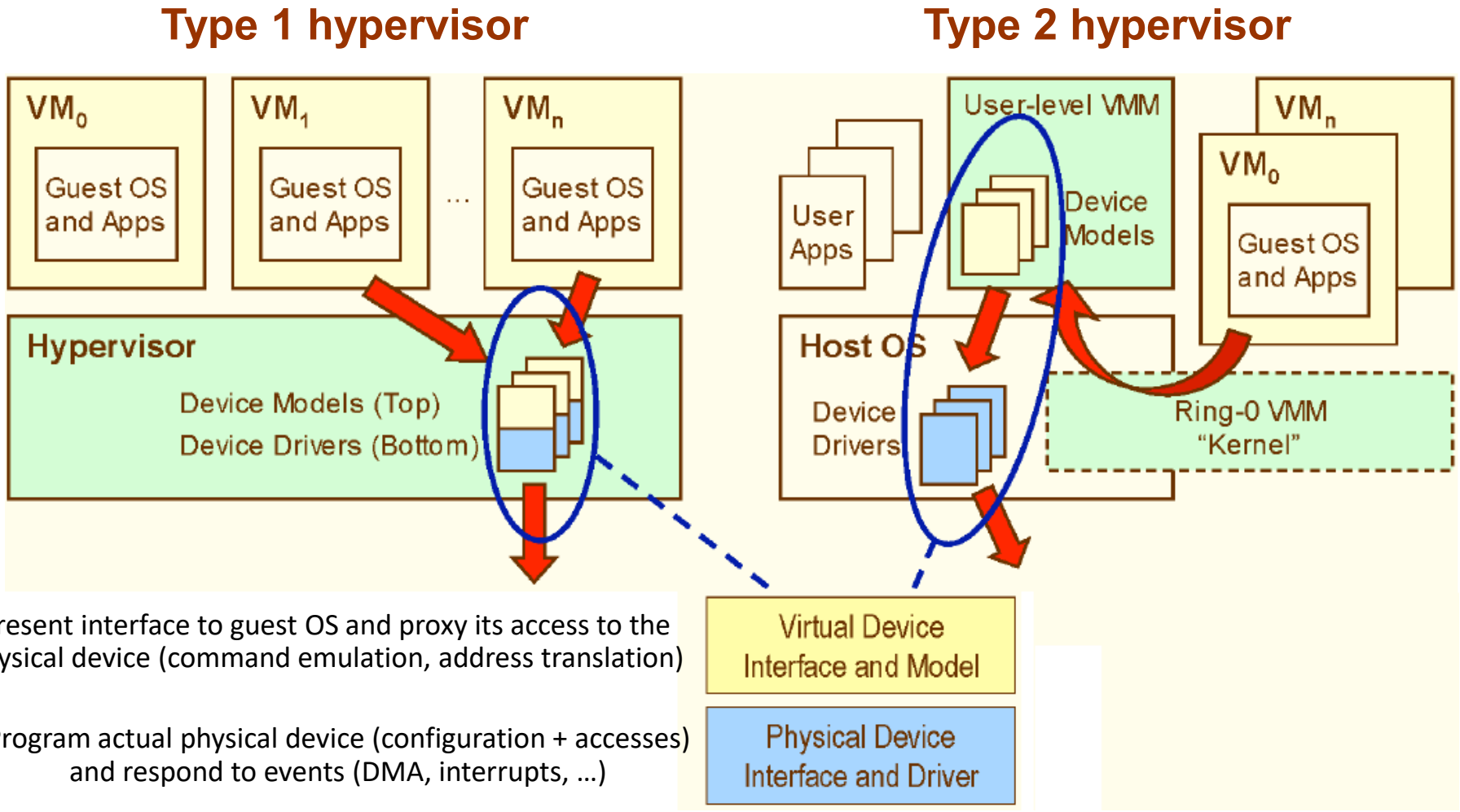
↑ Simple hypervisor

↓ Overhead due to calls and data redirection / world switch



Courtesy of [Chun17]

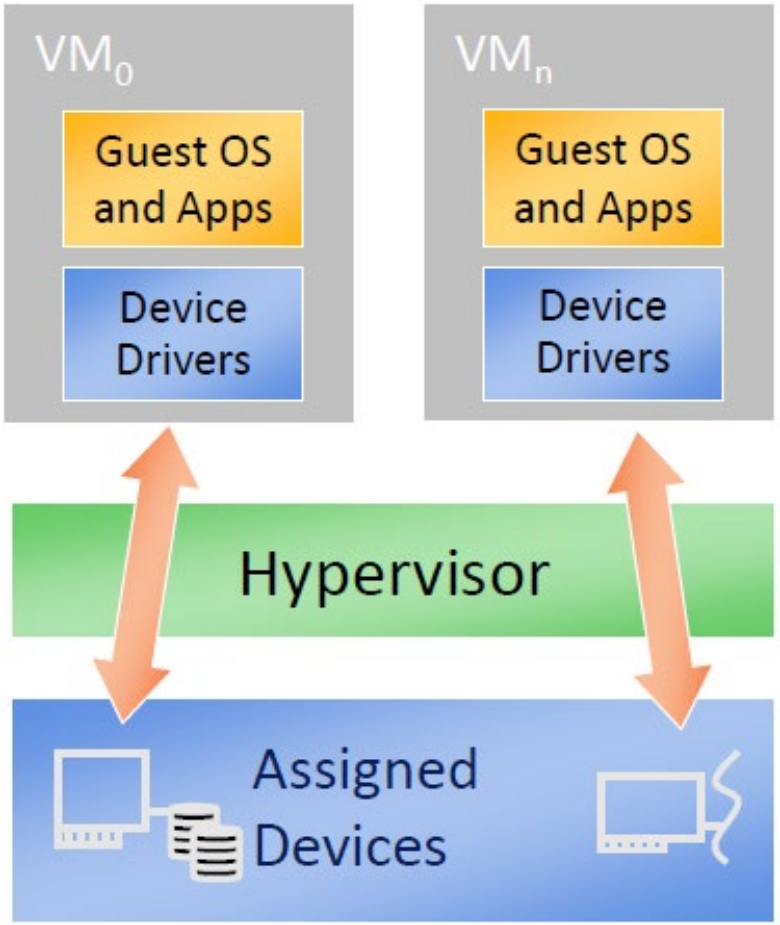
Virtual device front-end/back-end



Courtesy of [SmUh05]

Pass-through model

- Physical device is directly and exclusively assigned to a particular guest OS
- ↑ Removes emulation overhead: highest performance
- ↑ Simple hypervisor
- ↓ Strong coupling between guest OS and HW: hard VM migration, inflexible, hard device sharing

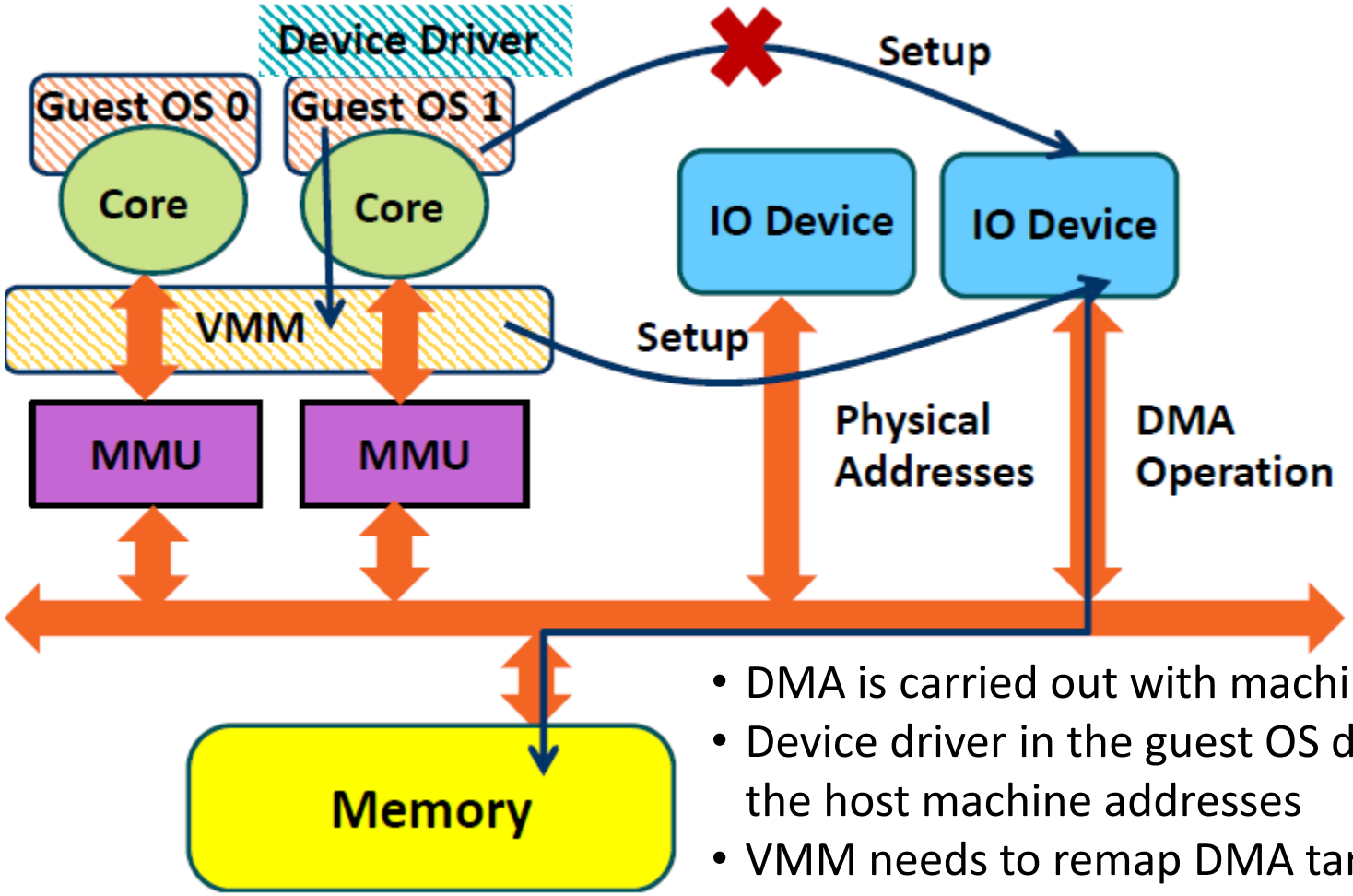


Courtesy of [Chun17]

Interception: Full virtualization

- Trap-and-emulate I/O operations of guest OS
  A. Memory-mapped I/O: Devices mapped in regular memory, accessed with load/store instructions
  ⇒ Map pages as not-present/read-only, so their access/modification by the guest traps to VMM
  B. Port-mapped I/O: Devices mapped in a separate address space, access with `in`/`out` instructions
  ⇒ `in`/`out` are privileged, so they trap to the VMM
  C. Direct Memory Access (DMA): devices can access memory bypassing the CPU
  ⇒ Address remapping is needed

Interception: DMA



- DMA is carried out with machine addresses
- Device driver in the guest OS does not know the host machine addresses
- VMM needs to remap DMA target (guest physical address to host machine address)

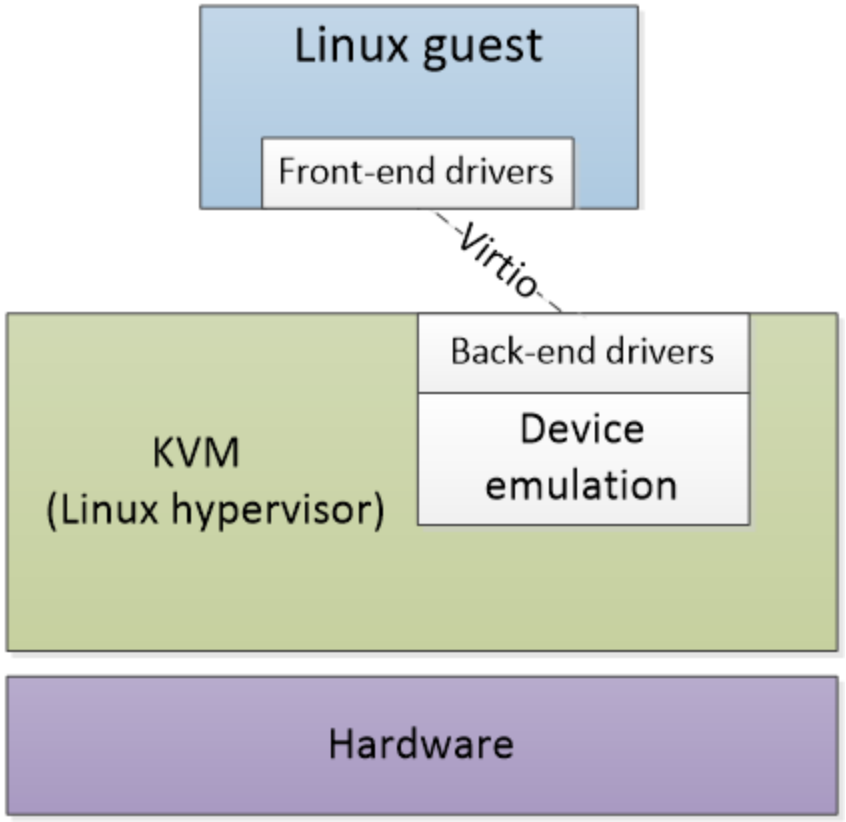
Courtesy of [BaKe16]

Interception: Paravirtualization

- Redesign virtual devices to optimize them for the virtualization layer esp. regarding interception
  A. Make hypercalls to VMM to initiate requests
  B. Buffers shared between guest OS and VMM

↑ Lower overhead

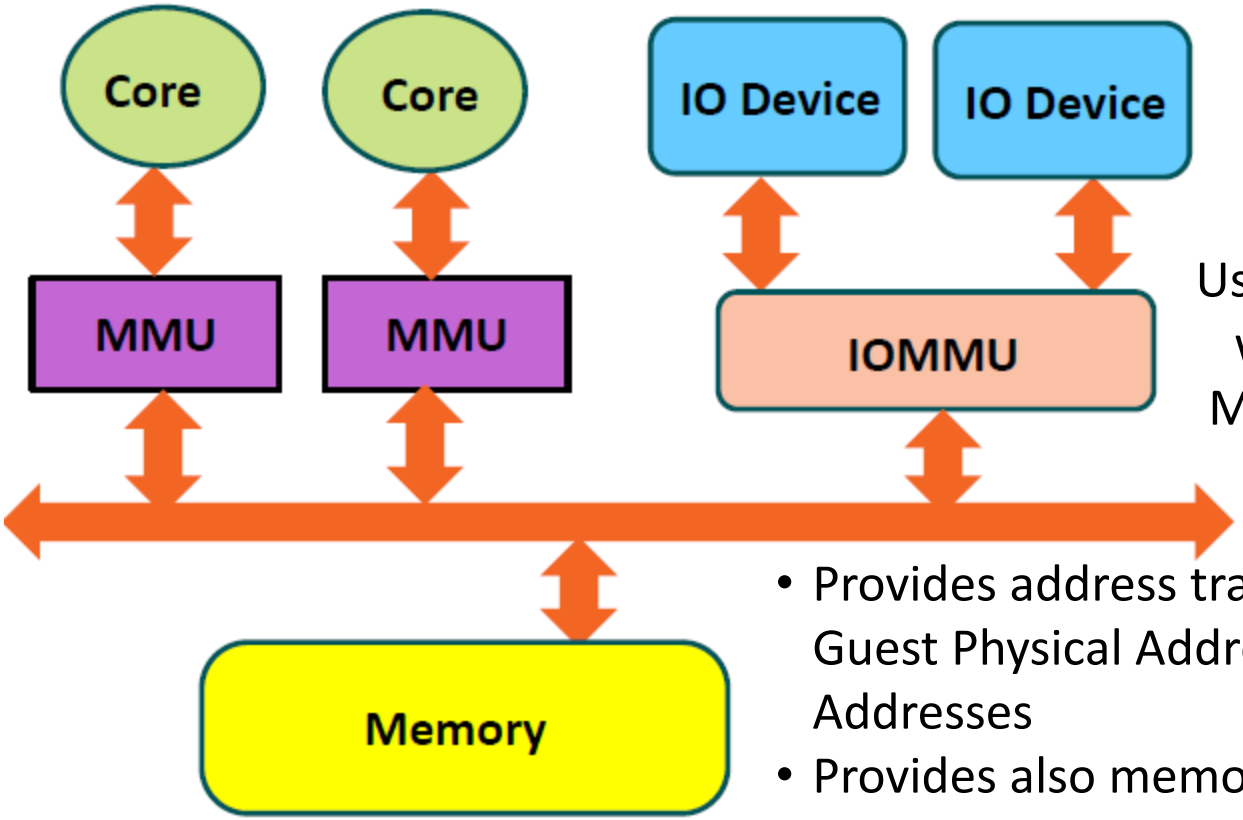
↓ Specific drivers for each guest OS (less portable)



e.g., **virt-io** (paravirtualized I/O framework for KVM/QEMU)

HW-assisted I/O virtualization

- VT-d (Intel)



Use of I/O MMU hardware which virtualizes I/O as MMU virtualizes memory

- Provides address translation for DMA: Guest Physical Addresses to Host Machine Addresses
- Provides also memory protection for DMA and dynamic remapping of interrupts

Courtesy of [BaKe16]

Contents

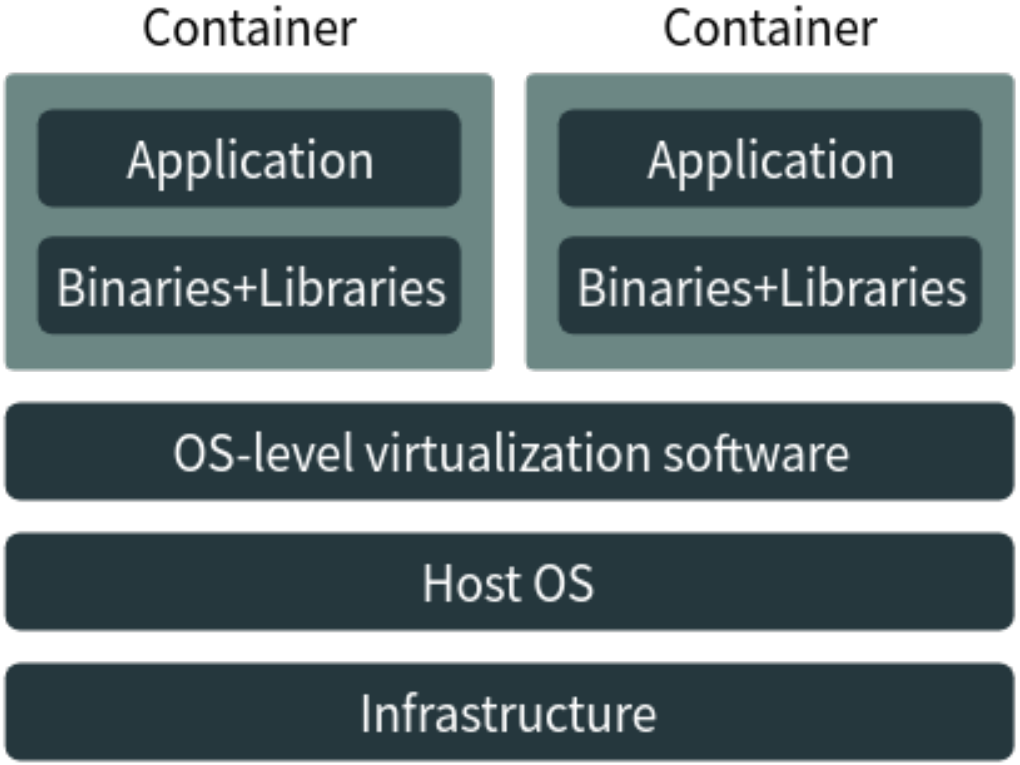
- Introduction
- Process scheduling
- Process resilience
- **Virtualization**
  - Introduction
  - Hardware virtualization
  - **OS-level virtualization**
  - Virtual clusters and resource management

OS-level virtualization

- HW virtualization drawbacks
  - ↓ Overhead to run VM instances due to emulation
  - ↓ Slow VM initialization due to VM creation and loading of guest OS
  - ↓ Great size of VM images and probable replication of contents (e.g. images with the same OS)
- If we just aim to manage the performance of applications and their resource use, and to be able to start, stop, move, and manage them easily, we should not need HW virtualization

OS-level virtualization

- Multiple isolated user-space instances (a.k.a. **containers**) share a single operating system



OS-level virtualization

- A container 'feels' like a virtual machine …
  - Each container has its own process space
  - Each container has its own network interfaces
  - A container can run stuff as root
  - A container can install packages
  - A container can get a shell on it
- … but all of them share the same OS
  - Isolation is achieved by changing what containers can see about the OS: it is a namespace game

OS-level virtualization

- Advantages
  - ↑ Little overhead (no emulation)
  - ↑ Minimal startup/shutdown
  - ↑ Low resource requirements and high scalability
- Drawbacks
  - ↓ All the containers in a machine MUST use the same OS (i.e. the host OS)
  - ↓ Containers cannot have their own modules
- Let's see some technologies used typically to achieve container isolation

Container isolation: namespaces

- **Namespaces** provide processes with their own view of the operating system
- There are several types of namespaces: pid, net, mnt, uts, ipc, user, time, cgroup
- A process belongs to a namespace of each type
- A new process can re-use none/all/some of the namespaces of its parent
- Namespaces are created with `clone` or `unshare` system calls, with flags `CLONE_NEW*`
- `setns` call allows joining an existing namespace
- A namespace is deleted when last process leaves

Container isolation: namespaces

1. **pid**: processes have their own PID numbering and can only 'affect' others in this namespace
2. **net**: processes get their own private network stack, including network interfaces, routing tables, iptables rules, sockets
3. **uts**: processes can have its own hostname
4. **mnt**: processes can have their own root file system and "private" mounts (e.g. /proc, /tmp)
5. **ipc**: processes can have their own message queues, semaphores, and shared memory

Container isolation: namespaces

6. **user**: maps process UIDs-GIDs outside and inside the namespace (e.g. privileged user ID 0 inside vs. unprivileged user ID outside)
7. **time**: processes can see a different clock (slower, faster, with an offset)
8. **cgroup**: processes have their own set of cgroup root directories

⇒ More info: http://man7.org/linux/man-pages/man7/namespaces.7.html

Container isolation: namespaces

- Example 1: 'UTS' namespace

```
$ hostname
osmiri-xx
$ sudo unshare --uts
$ hostname osmiri-yy
$ hostname
osmiri-yy
$ exit
$ hostname
osmiri-xx
```

create new 'UTS' namespace ↗

→ change hostname

→ namespace hostname has changed

→ computer hostname has not changed

- Example 2: 'mount' namespace

```
$ ls /tmp
...
[OUTPUT]
...
$ sudo unshare --mount
$ mount -t tmpfs none /tmp
$ ls /tmp
[EMPTY]
$ exit
$ ls /tmp
...
[OUTPUT]
...
```

create new 'mount' namespace ↗

→ set up /tmp

→ namespace has a private (empty) /tmp

→ computer /tmp has not changed

Container isolation: namespaces

- Example 3a: 'PID' namespace

create new 'PID' namespace

```
$ yes > /dev/null &
$ sudo unshare --pid --fork ↗
$ echo $$; yes > /dev/null & echo $!
1    → 'bash'    → namespace has its
10   → 'yes'       own PID numbers
$ ps -e  → 'ps' still sees all the processes
...           (with external PID numbers)
3624  pts/0  00:00:55  yes
3891  pts/1  00:00:17  yes
$ kill 3624 3891
-bash: kill: (3624) - No such process
-bash: kill: (3891) - No such process   → cannot kill external processes or
$ kill 10                                 using the external PID numbers,
[1]+  Terminated    yes > /dev/null       only through the internal PIDs
$ exit
```

- Example 3b: 'PID' & 'mount' namespaces

create new 'PID' namespace with a local mount to /proc

```
$ sudo unshare --pid --fork --mount-proc ↑
$ yes > /dev/null &
$ ps -e                                  → 'ps' only sees
  1  pts/1  00:00:00  bash                  namespace
10  pts/1  00:00:05  yes                   processes
11  pts/1  00:00:00  ps                    (with internal
$ exit                                     PID numbers)
```

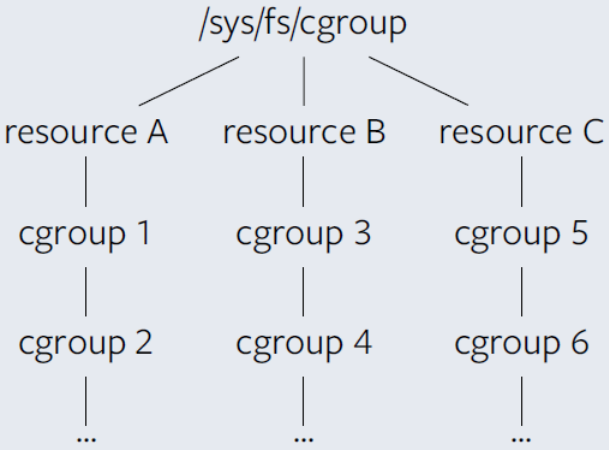
Container isolation: control groups

- cgroups provide **metering** and **limiting** of various types of resources (through specific controllers) for groups of processes
- Various controllers have been implemented: memory, CPU, block I/O, pids, freezer, ...
- Interface is via a pseudo-filesystem
  - cgroups manipulation via filesystem actions: mounting, create/remove directories, read/write files in them
- There are 2 (co-existing) cgroups versions
- More info: https://man7.org/linux/man-pages/man7/cgroups.7.html

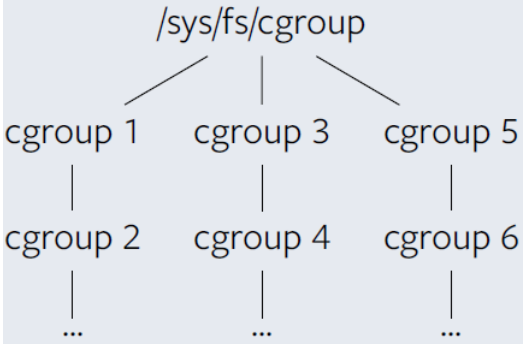
Container isolation: control groups

- cgroups are arranged in a hierarchy (a tree)

v1: one independent hierarchy per controller



v2: a single unified hierarchy

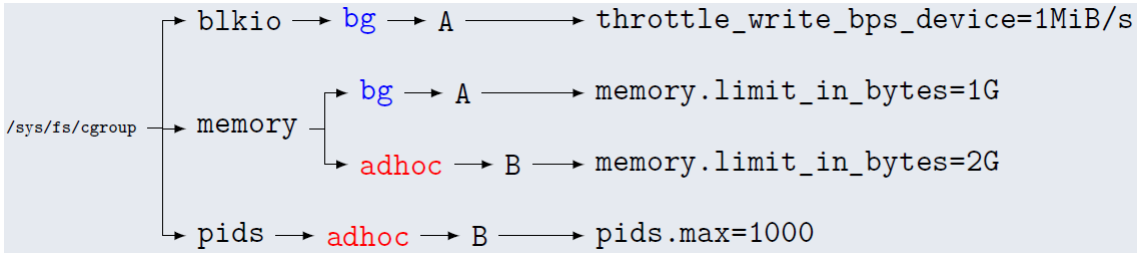


- New nodes (i.e. cgroups) can be created in each level
- Processes can be added/moved to nodes
  - Each process belongs to exactly one node (in each hierarchy)
  - Initially, it is placed in its parent's node(s)

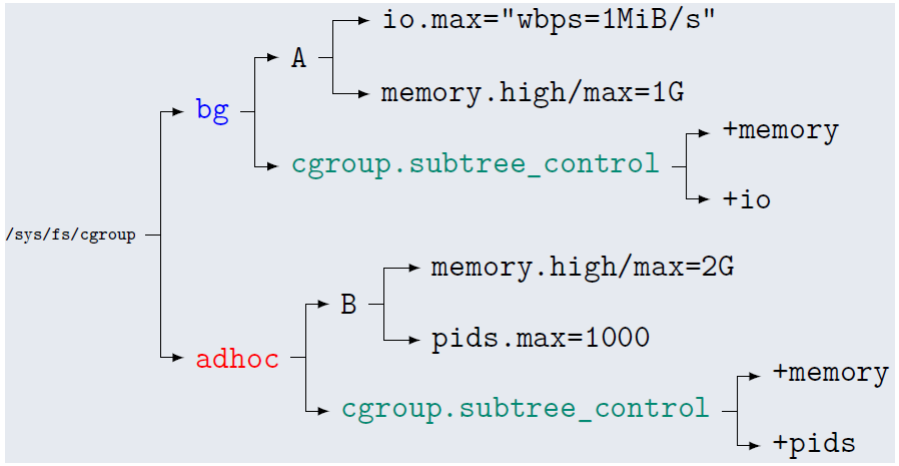
Container isolation: control groups

- Accounting and limits are applied per node (cgroup)
  - One can check the current usage of a node
  - One can associate limits to a node

v1:



v2:



Courtesy of [Down17]

Container isolation: control groups

- Example 1: 'cpu' cgroup

```
$ sudo mkdir /sys/fs/cgroup/cg1
$ sudo mkdir /sys/fs/cgroup/cg1/cg2
$ echo '+cpu' | sudo tee /sys/fs/cgroup/cg1/cgroup.subtree_control
$ echo '10000 100000' | sudo tee /sys/fs/cgroup/cg1/cg2/cpu.max
$ yes > /dev/null & sleep 1; ps -p $! -o %cpu -h; kill $!
97.0
$ bash
$ echo $$ | sudo tee /sys/fs/cgroup/cg1/cg2/cgroup.procs
$ yes > /dev/null & sleep 1; ps -p $! -o %cpu -h; kill $!
9.8
$ cat /sys/fs/cgroup/cg1/cg2/cpu.stat
usage_usec 517280
user_usec 144838
system_usec 372441
...
$ exit
```

→ create cgroups cg1 and cg2

→ enable 'cpu' controller for children of cg1

↘ define CPU usage limit: consume up to 10000 µs in each 100000 period

→ %CPU used by 'yes' command outside cg2

→ add 'bash' process to cg2

→ %CPU used by 'yes' command inside cg2

→ track the CPU time of processes in cg2

Container isolation: control groups

- Example 2: 'pids' cgroup

enable 'pids' controller for children of cg1 ↗

```
$ echo '+pids' | sudo tee /sys/fs/cgroup/cg1/cgroup.subtree_control
$ echo 5 | sudo tee /sys/fs/cgroup/cg1/cg2/pids.max → define limit on number of processes
$ bash
$ echo $$ | sudo tee /sys/fs/cgroup/cg1/cg2/cgroup.procs  → add 'bash' process to cg2
$ cat /sys/fs/cgroup/cg1/cg2/cgroup.procs
5108  → 'bash'
5150  → 'cat'
$ cat /sys/fs/cgroup/cg1/cg2/pids.current  → track the number of processes in cg2
2
$ for j in $(seq 1 5); do sleep 10 & done
[1] 5496
[2] 5497
[3] 5498                                   → cannot create more than 5 processes in cg2
[4] 5499
bash: fork: retry: Resource temporarily unavailable
$ exit
```

Container isolation: control groups

- Example 3: 'cpuset' cgroup

enable 'cpuset' controller for children of cg1

```
$ echo '+cpuset' | sudo tee /sys/fs/cgroup/cg1/cgroup.subtree_control ↗
$ cat /sys/fs/cgroup/cg1/cg2/cpuset.cpus.effective   → track CPUs granted to cg2 by cg1
0-1
$ echo 1 | sudo tee /sys/fs/cgroup/cg1/cg2/cpuset.cpus → define limit on CPUs to be used
$ yes > /dev/null & PID1=$!; yes > /dev/null & PID2=$!; sleep 1;
ps -p $PID1,$PID2 -o sgi_p -h; kill $PID1 $PID2
0
1    → outside cg2, each 'yes' command is running on a different CPU
$ bash
$ echo $$ | sudo tee /sys/fs/cgroup/cg1/cg2/cgroup.procs  → add 'bash' process to cg2
$ yes > /dev/null & PID1=$!; yes > /dev/null & PID2=$!; sleep 1;
ps -p $PID1,$PID2 -o sgi_p -h; kill $PID1 $PID2
1
1    → inside cg2, both 'yes' commands are running on CPU #1
$ exit
```

Container isolation: security

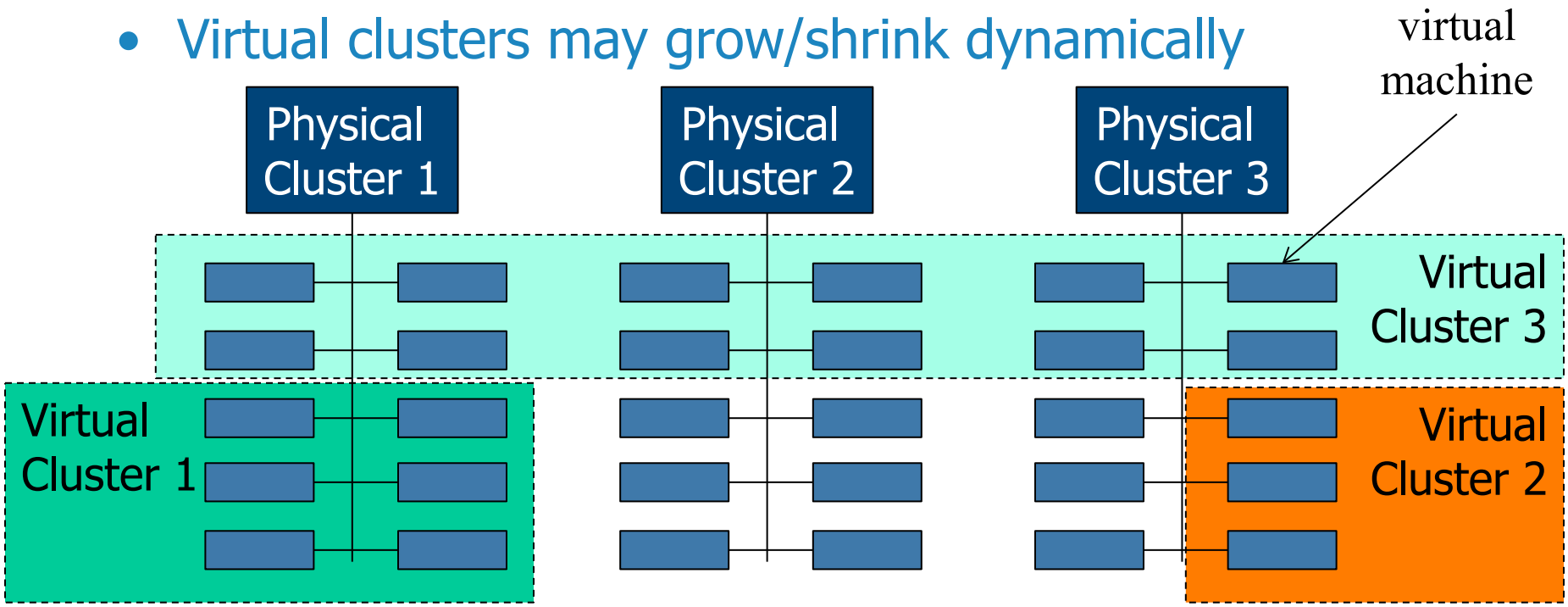
- Capabilities: Divide the privileges traditionally associated with root into distinct units, which can be independently enabled and disabled
  - A process with root privilege can be limited to get only the minimal permissions it needs to operate
  - More info: https://man7.org/linux/man-pages/man7/capabilities.7.html
- Seccomp: Define which system calls are allowed to be executed by a process
  - More info: https://man7.org/linux/man-pages/man2/seccomp.2.html

Contents

- Introduction
- Process scheduling
- Process resilience
- **Virtualization**
  - Introduction
  - Hardware virtualization
  - OS-level virtualization
  - **Virtual clusters and resource management**

Virtual clusters

- Built with VMs from one or more physical clusters
- VMs in a virtual cluster are interconnected logically by a virtual network
- Virtual clusters may grow/shrink dynamically



Courtesy of [HwDF11]

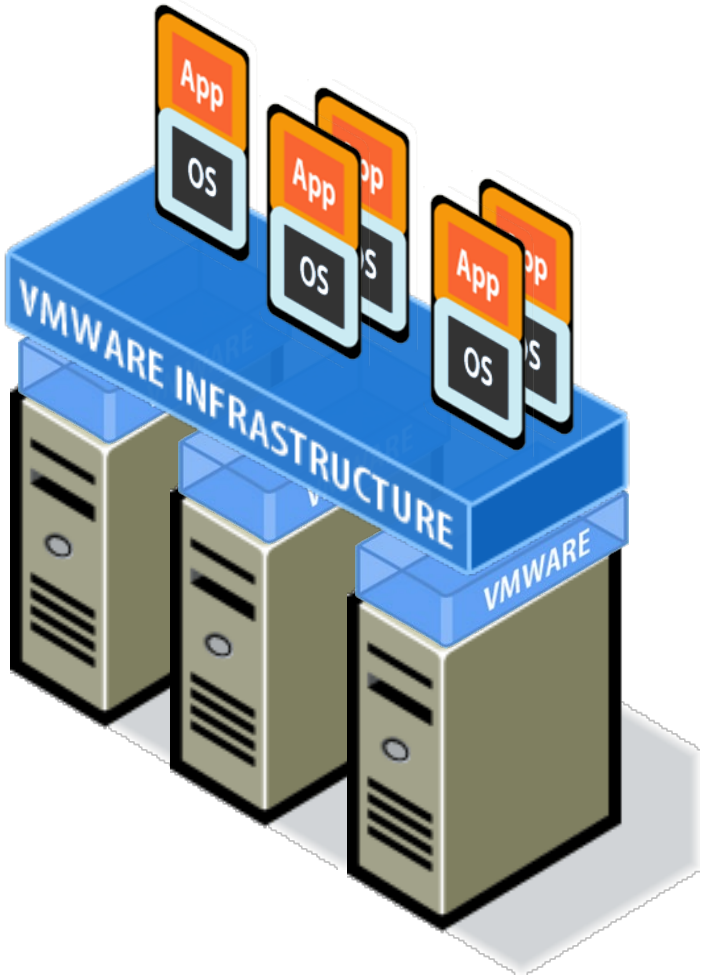
Virtual clusters challenges

- Host and VM monitoring and management
  - VM deployment
  - Live VM migration
  - VM allocation
  - VM load balancing
  - VM consolidation
  - VM elasticity
  - Fault tolerance
    - VM checkpoint/recovery
    - VM replica failover
  - VM scheduling (see 'vCPU scheduling')

VM deployment

- Build a complete guest system and distribute it to a physical host within the cluster as fast as possible





Courtesy of [Chun17]

VM deployment

1. Prepare the disk image
   - Use template images (including a preinstalled OS and certain application software)
   - Use Copy-on-Write (CoW) to avoid copying the whole raw image file
2. Configure the VM
   - Configure UUID, VM name, disk image, network settings, and allocated CPU and memory
3. Choose the destination host $\Rightarrow$ VM allocation
   - Fulfill the VM requirements while balancing load, consolidating VMs, reducing energy consumption, …

Live VM migration

- Allows moving a running guest from one physical host to another
- Requirements:

1. Negligible downtime
2. Lowest network bandwidth consumption
3. Reasonable total migration time
4. Do not disrupt other active services in host

➢ Must migrate memory, storage, and network

Memory migration

- Memory size: 100s MBs → few GBs
- Naïve approach: Suspend & Copy & Resume
  - But very high downtime!!
- Better approach… minimize 'Suspend'
- Two mechanisms:
  1. Pre-copy mechanism
  2. Post-copy mechanism
- Additionally, memory can be compressed to reduce network bandwidth

Pre-copy migration steps

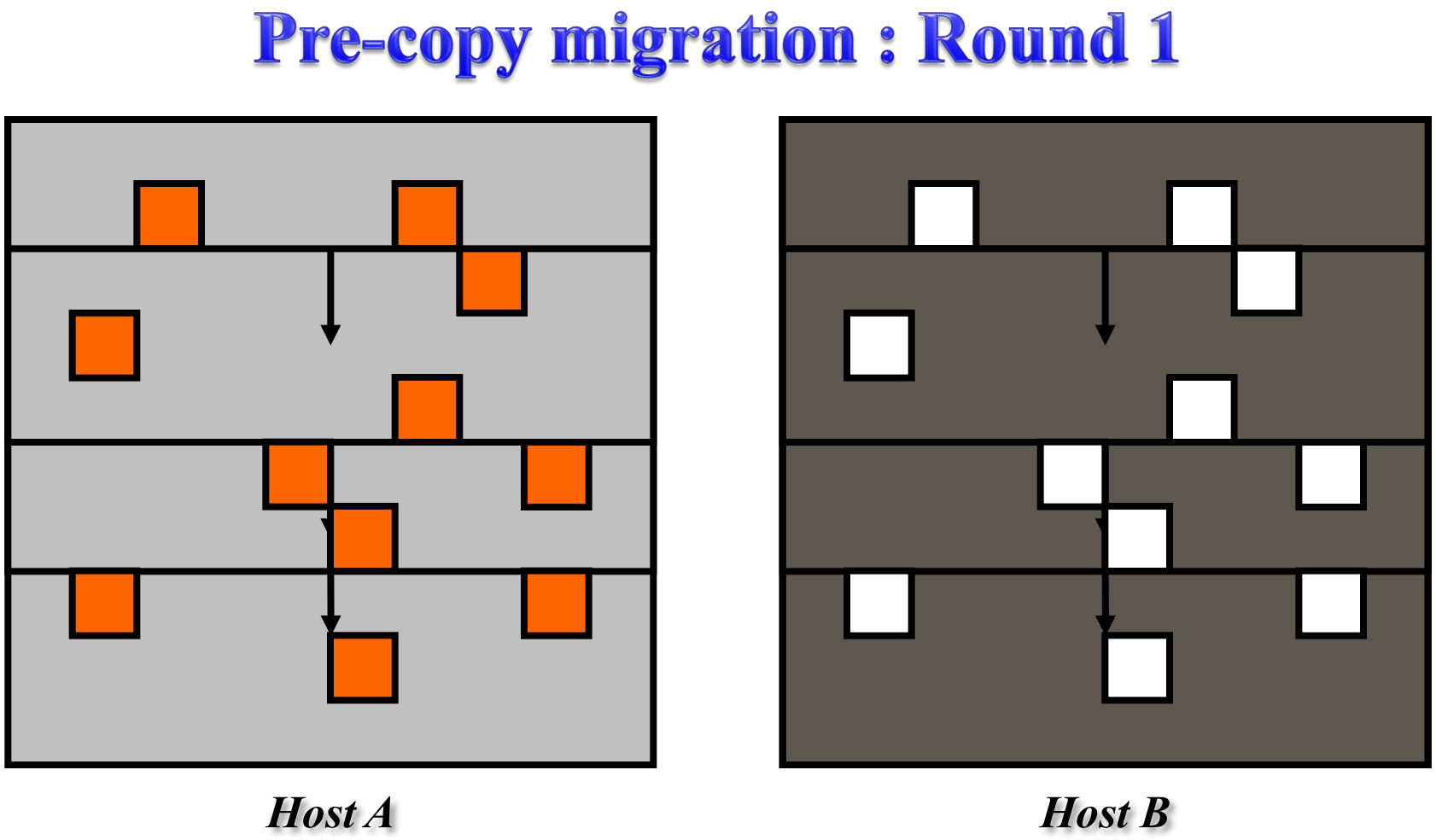
1. Pre-migration + reservation
   - Determine migrating VM
   - Determine destination host

   ⇒ VM allocation
   - Initialize a VM on the target host
2. Memory push (iterative pre-copy)
   - Transfer all memory pages in the first round
   - Copy dirty memory pages (data which is changed) in successive rounds
   - Repeat until the number dirty pages becomes smaller than a threshold

Pre-copy migration steps

3. Stop and copy
   - Suspend VM on source host
   - Network connection is redirected to the new VM
   - Transfer the remaining dirty pages
     - Transfer VCPU and network states as well
4. Commitment + activation
   - VM reloads state and resumes its execution on the destination host
   - Remove original VM from the source host

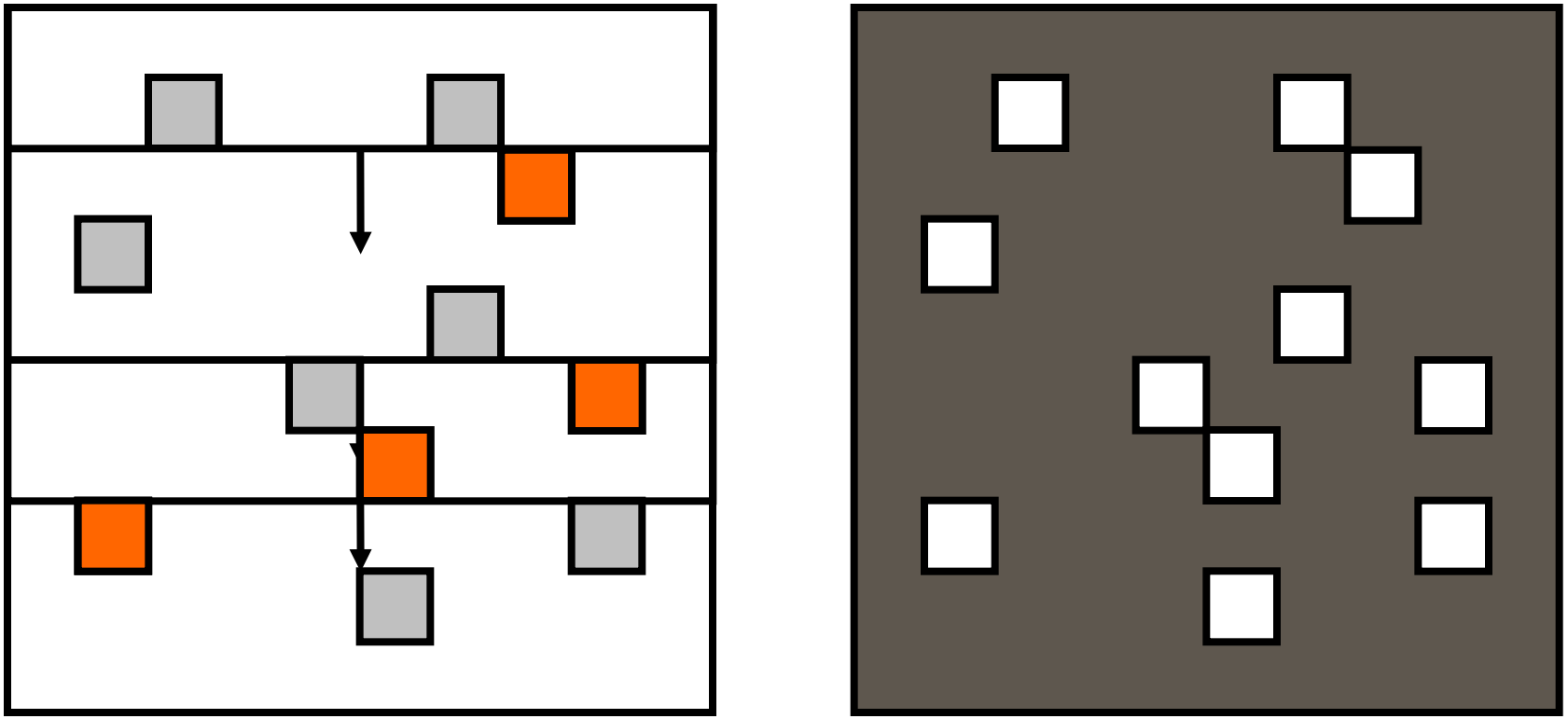
Pre-copy migration



Courtesy of [Chun17]

Pre-copy migration

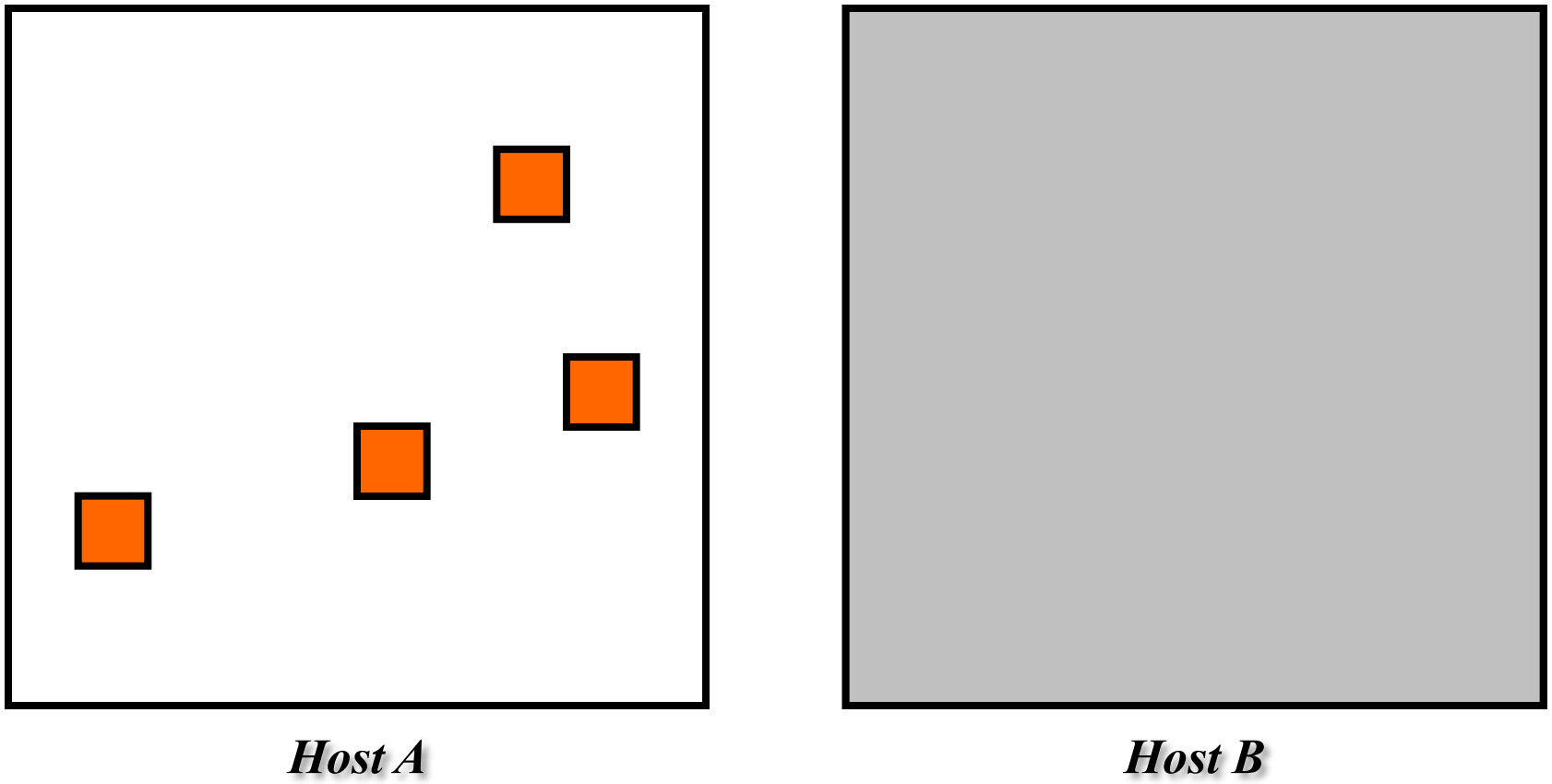
Pre-copy migration : Round 2



Courtesy of [Chun17]

Pre-copy migration

Stop and copy : Final Round



Courtesy of [Chun17]

Pre-copy migration

- Advantages
  - ↑ No need to fetch pages from the source host
  - ↑ An up-to-date state of the VM is retained at the source host during migration ⇒ If the destination host fails during migration, VM can be recovered
- Drawbacks
  - ↓ Migration may never end if source host dirties pages faster than they can be transferred
  - ↓ Can transfer the same page multiple times if the page is dirtied repeatedly at the source host during migration

Post-copy migration steps

1. Pre-migration+reservation remain the same
2. Stop and copy
   - Suspend VM and transfer minimal execution state
3. Activation: resume VM at destination host
4. Running VM
   - During the execution of the VM, its memory pages are pushed from source host
     - If VM accesses a not yet received page, it is faulted in from the source over the network (remote page fault)
   - Remove original VM from the source host when all the pages have been transferred

Post-copy migration

- Advantages
  - ↑ Live migration ends in a finite time
  - ↑ Sends each page exactly once over the network
- Drawbacks
  - ↓ VM state is distributed over both source and destination hosts ⇒ If the destination host fails during migration, the VM cannot be recovered
  - ↓ Latency of fetching pages from the source host

Storage migration

- Each VM must have a consistent, location-independent view of the file system
  a) Virtual disks reside on the same shared volume accessible by all the hosts
     ↑ No copy needed
     ↓ Requires HW or SW support for shared storage
  b) Shared-nothing migration: virtual disks are also moved across different storage elements
     ↑ Can work with any kind of storage support
     ↓ Must be optimized to avoid huge downtimes due to the current trend on disk capacity
     - e.g. Snapshotting, Dirty Block Tracking, IO Mirroring

Storage migration

A. Snapshotting
   1. Copy base disk while the VM keeps working in a delta disk (storing an incremental snapshot)
   2. After the copy, consolidate delta disk into base disk. VM keeps working in another delta disk
   3. Repeat the process until the amount of data in the delta disk becomes smaller than a threshold
   4. Suspend VM and consolidate the final delta disk

B. Dirty Block Tracking
   – Same procedure than memory pre-copy but copying disk blocks instead of memory pages

Storage migration

C. IO Mirroring

- Consists of mirroring all new writes from the source to the destination concurrently with a bulk copy of the base disk

1. Writes to a region that has already been copied are mirrored to the source and destination
2. Writes to the region currently being copied are deferred until that copy finishes and then they are mirrored at both hosts
3. Writes to a region that will be copied later are issued to the source only

Network migration

- Maintain all open network connections
- VM keeps its virtual MAC and IP addresses

a) If migrating within the same local subnet
   - Source host broadcasts ARP reply advertising that the IP has moved to a new location
     - Some in-flight packets may be lost, but TCP retransmissions will take care of them

b) Otherwise, an indirection layer must exist
   - e.g. Mobile IP: a node on the source network forwards packets to a care-of address on the destination network

VM allocation

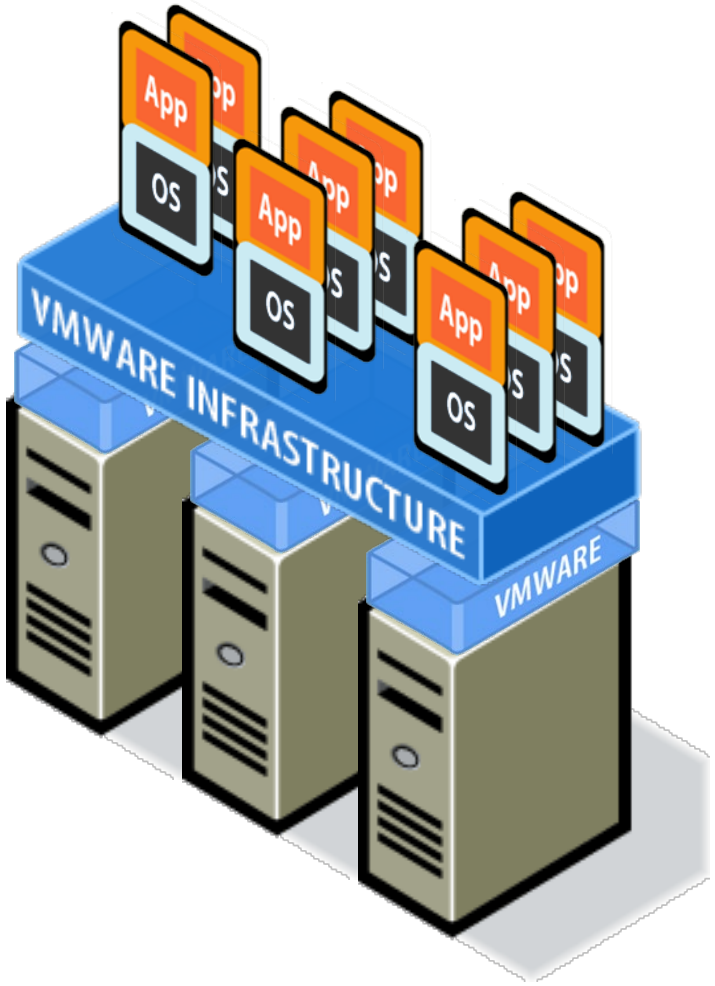
- Decide mapping of VMs to physical hosts
- Migrations allow moving VMs among hosts
- Can be viewed as a (fully dynamic) packing problem, which is known to be NP-hard
- Simple bin-packing heuristics have been adapted to the VM allocation problem
  - e.g., First Fit and Best Fit packing heuristics (see previous lesson)
- But VM allocation is harder than bin-packing

VM allocation vs. bin-packing

1. Several resource dimensions (CPU, memory, disk) define the hosts capacities and VM sizes
2. The performance of a physical host degrades when it is overloaded (or close to)
3. Physical hosts can have different capacities
4. The number of hosts is finite, constraining the number of VMs that can be allocated
5. The number of VMs and the number of available hosts are volatile (may change over time)
6. VM migrations incur a performance cost

VM consolidation

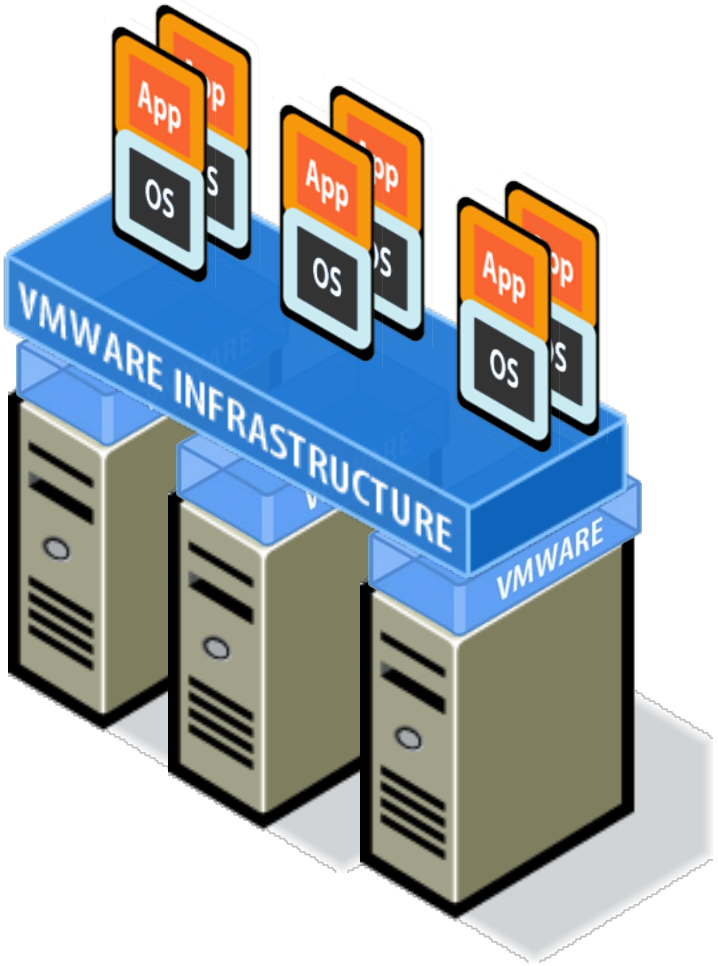
- Consolidate VMs running on under-utilized hosts
- Live VM migration supports dynamic consolidation
- Enhances utilization
- Suspend idle hosts to lower operational costs (e.g. electricity)



Courtesy of [Chun17]

VM load balancing

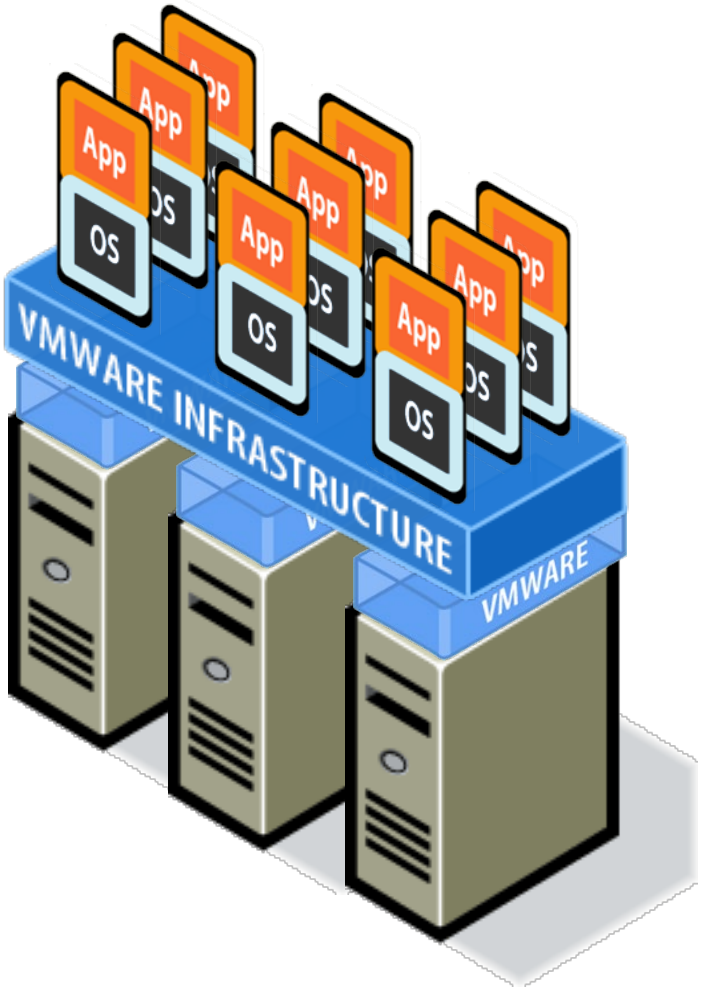
- Balance the VMs across hosts according to their changing resource requirements
- Spread the VMs to foster parallelism and dependability
- Live VM migration supports dynamic load balancing



Courtesy of [Chun17]

VM elasticity

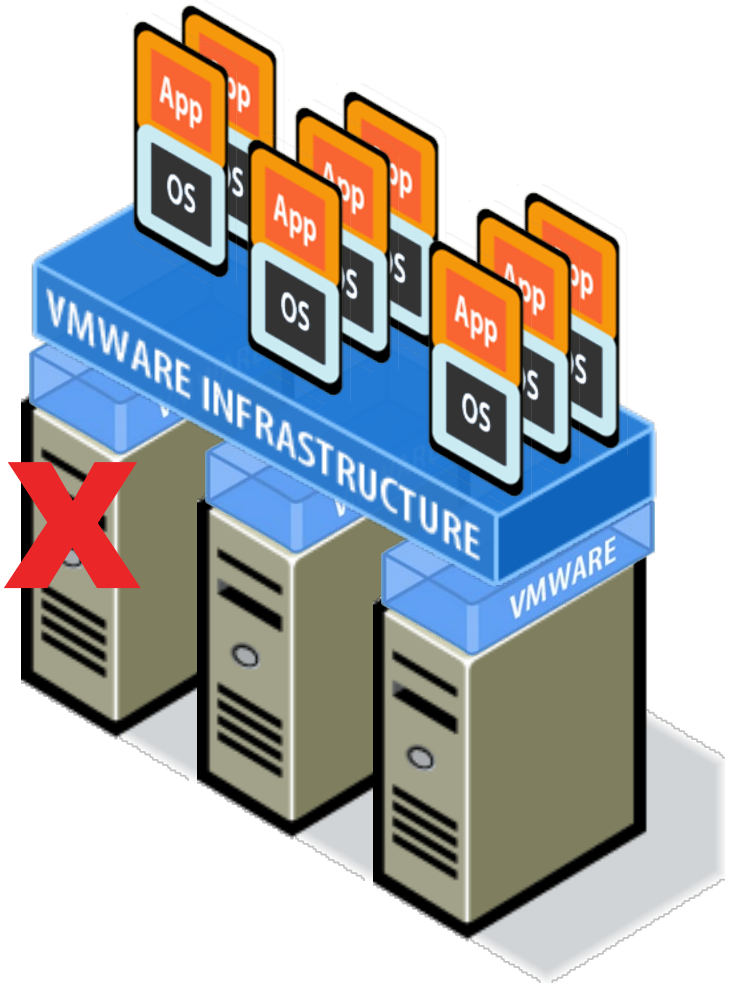
- Dynamically and transparently scale up/down VMs that provide a service
- Horizontal: increase number of VMs among hosts
- Vertical: increase the size of the VM



Courtesy of [Chun17]

VM checkpoint/recovery

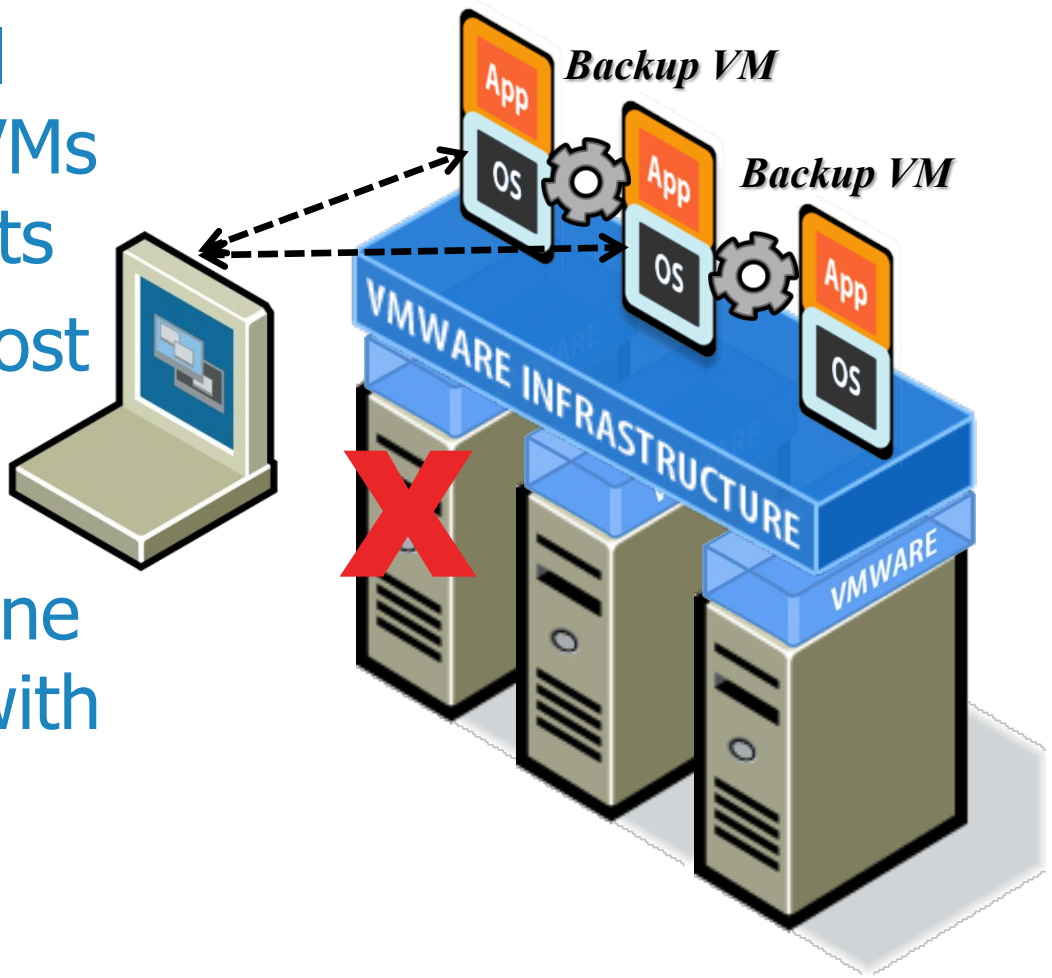
- Upon a host failure, VMs are automatically resumed [from their last checkpoint] on other physical hosts
- Live VM migration allows proactively moving VMs when a failure seems likely in the short term



Courtesy of [Chun17]

VM replica failover

- Maintain several replicas of the VMs on different hosts
- Upon a VM or host failure, service is automatically resumed from one of the replicas with no downtime



Courtesy of [Chun17]

References

[TaBo23] A.S. Tanenbaum and H. Bos. 2023. Modern Operating Systems, 5th ed. Pearson Education, Inc., Hoboken, NJ, USA

[TaSt23] A.S. Tanenbaum and M. van Steen. 2023. Distributed Systems: Principles and Paradigms, 4th edition, distributed-systems.net

[SiGG18] A. Silberschatz, P.B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley Publishing, 2018

[HwDF11] K. Hwang, J. Dongarra, G. Fox; Distributed and Cloud Computing, 2011

[Chun17] Yeh-Ching Chung, Cloud Computing (CS5421)

[BaKe16] A. Basu, A.G. Kegel, IOMMU: Virtualizing IO through the IO Memory Management Unit, http://pages.cs.wisc.edu/~basu/isca_iommu_tutorial/index.htm

[Down17] Chris Down, cgroupv2: Linux's new unified control group system, FOSDEM 2017, https://archive.fosdem.org/2017/schedule/event/cgroupv2/

[SSCZ13] X. Song, J. Shi, H. Chen, and B. Zang. Schedule processes, not VCPUs. 4th Asia-Pacific Workshop on Systems (APSys'13). Singapore, July 29 - 30, 2013

[Jone10] M. Jones, Virtio: An I/O virtualization framework for Linux, 2010

[SmNa05] J.E. Smith and R. Nair, The architecture of virtual machines, Computer, 38 (5), pp. 32-38, May 2005

References

[SmUh05] System Virtual Machines, HotChips 17 Tutorial, 2005, Intel Corporation

[DuCh99] K.J. Duda and D.R. Cheriton. Borrowed-virtual-time (BVT) scheduling: supporting latency-sensitive threads in a general-purpose scheduler. SIGOPS Oper. Syst. Rev. 33, 5 (Dec. 1999), pp. 261-276

[SuVL01] J. Sugerman, G. Venkitachalam, and B.-H. Lim. Virtualizing I/O Devices on VMware Workstation's Hosted Virtual Machine Monitor. 2001 USENIX Annual Technical Conference, June 25-30, 2001, Boston, MA, USA, pp. 1-14